

G-ThinkerCG: A Unified CPU-GPU Framework for Efficient Parallel Subgraph Finding

Akhlaque Ahmad, Da Yan, Lyuheng Yuan, Zhijun Zhao, Yujie Lu

Indiana University Bloomington

{akahmad, yanda, lyyuan, zzh10, yujlu}@iu.edu

ABSTRACT

Subgraph finding identifies all subgraphs of a large graph G that satisfy given criteria. As a core primitive for analyzing social, biological, and knowledge graphs, subgraph finding remains challenging on heterogeneous CPU-GPU platforms due to irregular search spaces, load imbalance, and strict GPU memory limits.

We present G-ThinkerCG, a unified parallel framework for efficient subgraph finding on heterogeneous CPU-GPU platforms. It employs a subgraph-task abstraction to coordinate recursive CPU search with massively parallel GPU exploration, and supports bidirectional work stealing between CPU and GPU. On the GPU, a hybrid BFS-DFS subgraph extension strategy with host buffering ensures bounded device memory usage while maximizing parallelism, whereas the CPU effectively handles straggler tasks with deep search trees through automatic task decomposition. Experiments on real-world graphs confirm that hybrid CPU-GPU execution excels when search trees have irregular depths, outperforming GPU-only execution by up to 8.6 $\times$; for GPU execution alone, our BFS-DFS hybrid strategy beats prior SOTA by up to 2.7 $\times$.

PVLDB Reference Format:

Akhlaque Ahmad, Da Yan, Lyuheng Yuan, Zhijun Zhao, Yujie Lu.
G-ThinkerCG: A Unified CPU-GPU Framework for Efficient Parallel Subgraph Finding. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/akhlaqueak/G-ThinkerCG>.

1 INTRODUCTION

Given a large graph $G = (V, E)$, subgraph finding (SF) enumerates or counts subgraphs of G that satisfy application-specific conditions. SF is a fundamental primitive in social, biological, and knowledge graph analytics [20]. The search space is combinatorial and typically traversed via recursive backtracking over a redundancy-free subgraph enumeration tree; practical algorithms, therefore, rely on aggressive pruning and careful scheduling to keep search tractable.

CPU and GPU solutions to SF exploit different hardware strengths. **BFS-style GPU** subgraph extension enables coalesced memory access and high throughput, but requires materializing all intermediate subgraphs per level, so can run out of memory (OOM);

G^2 -AIMD [43] mitigates this via adaptive chunk sizing, yet its overflow handling discards computation on failed chunks, wasting compute work. **CPU** approaches are usually **DFS-style** hence memory-efficient: they adopt recursive task-centric subgraph enumeration with dynamic ‘big’ task decomposition (e.g., timeout-based) to handle deep and irregular search tree branches [20, 40, 44], but this model is difficult to execute efficiently on SIMT hardware.

Memory-efficient **DFS-style GPU** approaches exist [36, 45] where each warp explores a subgraph enumeration branch using a compact stack; however, they require preallocated stack space in device memory and are only practical when the search depth is bounded by a known number. **Hybrid BFS-DFS** strategies exist on **CPUs** [10], but porting them to GPU is non-trivial due to limited device memory and unknown extension depths.

Moreover, SF workloads can be inherently skewed: in power-law graphs, most vertices spawn shallow, similar-depth subgraph enumeration trees with abundant parallelism, whereas a few vertices lead to exceptionally deep, straggler-prone trees. This asymmetry suggests a natural division of labor—GPUs excel at processing many tasks with comparable depths via massively parallel level-wise extension, while CPUs are better for a small number of extremely deep trees using recursive DFS. Despite this complementarity, effective CPU-GPU co-processing for exact SF remains unexplored.

We introduce **G-ThinkerCG**, a unified subgraph-centric framework for scalable parallel SF on heterogeneous CPU-GPU platforms. G-ThinkerCG adopts a *subgraph-task* abstraction that treats a subgraph and its extension subtree as the smallest schedulable unit, which unifies recursive CPU backtracking search with GPU level-wise extensions, and enables bidirectional work stealing between CPU threads and GPU warps. On the GPU, a hybrid BFS-DFS extension strategy bounds per-level expansion without requiring maximum depth knowledge, avoiding unbounded memory growth. To handle the cases when device memory is exhausted, a lightweight host-side buffering mechanism is introduced to transparently spill and resume subgraph-tasks. Together with a scheduling policy that routes abundant shallow-depth tasks to the massively parallel GPU and the few deep, irregular ones to the CPU, these mechanisms enable efficient, memory-bounded co-processing.

Our main contributions are summarized as follows:

- G-ThinkerCG is designed as a general framework for efficient co-processing of irregular SF workloads across CPU and GPU, following a unified *subgraph-task* abstraction.
- A hybrid BFS-DFS subgraph extension strategy is designed for GPU execution with bounded device memory. It removes maximum depth knowledge required by prior works, by supporting host-side subgraph buffering that transparently spills GPU tasks to host memory and resumes them.
- We advocate a degeneracy-ordering-based scheduling policy to route abundant shallow-depth tasks to the massively

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Algorithm 1 Bron-Kerbosch Algorithm for Maximal Cliques

```
1: BRONKERBOSCH( $\emptyset, V, \emptyset$ )
2: procedure BRONKERBOSCH( $R, P, X$ )
3:   if  $P = \emptyset$  and  $X = \emptyset$  then output  $R$ 
4:   for all  $v \in P$  do
5:     BRONKERBOSCH( $R \cup \{v\}, P \cap N(v), X \cap N(v)$ )
6:      $P \leftarrow P \setminus \{v\}$ 
7:      $X \leftarrow X \cup \{v\}$ 
```

parallel GPU and the few deep, irregular ones to the CPU, to fully harness complementary hardware strengths of CPU and GPU, while supporting CPU-GPU work stealing.

- We implement three representative SF applications on G-ThinkerCG, and conduct extensive experiments on real-world graphs which confirm that hybrid execution excels when search depths are irregular (e.g., cohesive subgraph mining), while our hybrid BFS-DFS strategy on GPU already outperforms prior state-of-the-art GPU frameworks.

In the sequel, Section 2 reviews preliminaries and related work. Section 3 presents our hybrid BFS-DFS extension strategy on the GPU. Sections 4 and 5 introduces the system design and programming interface of G-ThinkerCG, respectively. Finally, Section 6 describes the implemented applications on G-ThinkerCG, Section 7 reports the performance results, and Section 8 concludes.

2 PRELIMINARIES AND RELATED WORK

We review the GPU and SF concepts relevant to our system design, followed by a discussion of related work on parallel SF systems.

2.1 Preliminaries

CUDA Model. GPU kernels are executed by many threads organized into blocks, which are further divided into warps of 32 threads. A warp is the smallest scheduling unit and executes instructions in a lockstep manner; divergent control flow within a warp is serialized. Due to the GPU memory hierarchy and warp-level execution model, achieving coalesced memory access is critical for performance.

In GPU-based SF systems, warps typically process adjacency lists or vertex sets collaboratively to ensure coalesced memory access. GPU threads cannot directly access host memory; data must reside in device memory. These features strongly influence how subgraph enumeration and task scheduling are implemented on GPUs.

Subgraph Finding (SF). SF algorithms perform recursive backtracking over a *subgraph enumeration tree*, where each node represents a subgraph instance and its descendants correspond to all valid extensions of that subgraph. The enumeration tree is redundancy-free: each subgraph is generated exactly once. For a real power-law graph, the fanout and depth of the enumeration trees extended from different vertices can vary significantly.

We illustrate by considering the Bron-Kerbosch (BK) algorithm [6] as shown in Algorithm 1, which recursively enumerates all maximal cliques in a graph. Specifically, BK maintains three disjoint vertex sets: (1) R , containing vertices in the current subgraph; (2) P , the set of candidates that can extend R into a larger clique; and (3) X , the exclusion set used to ensure maximality. Figure 1 illustrates the subgraph-enumeration recursion tree for a graph with four vertices $\{a, b, c, d\}$, where each node is annotated with the set R and corresponds to a subgraph of G induced by R . Let $N(v)$ denote the set

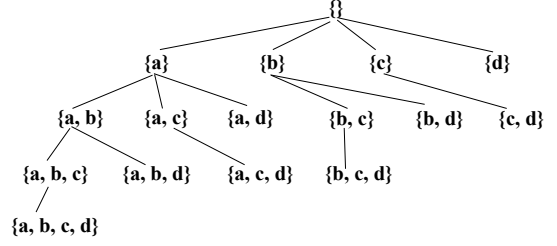


Figure 1: Set-Enumeration Tree

of neighbors of vertex v . The algorithm ensures that every vertex in P and X is adjacent to all vertices in R , as enforced via Line 5 which prunes non-neighbors when a new vertex v is added to R . The exclusion set X tracks those candidates previously explored in sibling branches. For example, at node $\{a, c\}$, vertex b has already been explored in the sibling branch $\{a, b\}$ and is therefore placed in X , preventing any clique still extendable by b from being reported as maximal. When P and X are both empty (Line 3), there are no further vertices that can be added to R , so R is a maximal clique.

This set-enumeration process is also widely adopted by SF algorithms for finding maximum clique [39, 40], maximum k -plex [4, 7, 17], maximal k -plexes [12, 15, 35], maximum quasi-clique [9], maximal quasi-cliques [21, 24, 25], and size-bounded community [41].

Another example of SF is subgraph matching (SM), which enumerates all subgraphs of a data graph G that are isomorphic to a query graph G_q . Many SM algorithms follow Ullmann’s backtracking framework [33], which recursively extends partial mappings while pruning infeasible candidates. The process also searches over a subgraph enumeration tree. See Appendix A [1] for more details.

Subgraph-Tasks. For a node S in the subgraph enumeration tree, we refer to the work of examining S and all valid extensions rooted at S as a *subgraph-task*, denoted by T_S . Each subgraph-task corresponds to a subtree of the enumeration tree and forms a natural unit for structuring and scheduling work in parallel SF systems. The initial tasks are given by those rooted directly below the enumeration-tree root, such as $T_{\{a\}}$, $T_{\{b\}}$, $T_{\{c\}}$, and $T_{\{d\}}$ in Figure 1.

2.2 Related Work

Vertex-centric systems such as Pregel-like systems [13, 18, 26, 27, 37] and their GPU variants [13, 18, 26, 37] conduct iterative computation and message passing on individual vertices. They primarily target iterative algorithms such as BFS and PageRank [38], so are not suitable for SF workloads that involve recursive exploration of subgraph-enumeration trees, motivating subgraph-centric models that operate on subgraph instances rather than individual vertices.

Earlier subgraph-centric SF systems [11, 16, 22, 23, 32, 34] provide abstractions for incrementally growing and filtering subgraphs. Among these, Pangolin [11] and GAMMA [22] support GPU execution. However, these systems extend subgraphs naïvely leading to device memory overflow or expensive out-of-core joins, and they often require explicit graph isomorphism checks to avoid generating duplicate subgraphs. Backtracking-based SF systems such as G-thinker [39] and its variants [8, 20, 24, 40, 44] allow SF algorithms to closely follow their serial counterparts, by conducting memory-efficient recursive subgraph backtracking for redundancy-free subgraph enumeration, and splitting a long-running enumeration task to avoid stragglers. However, these systems are all CPU-oriented.

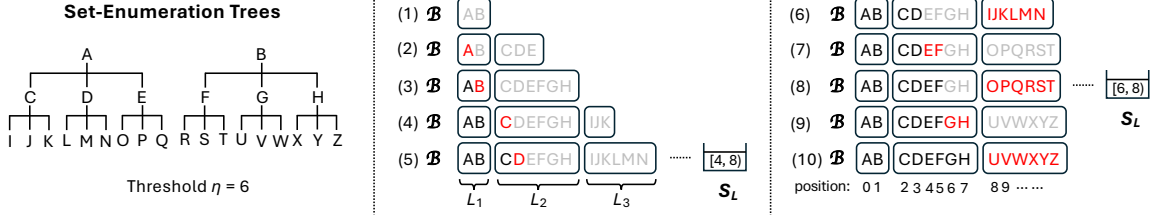


Figure 2: Illustration of Hybrid BFS-DFS Subgraph Extension

While most current GPU-based SF solutions are tailored to individual problems, G²-AIMD [43] is designed as a GPU framework for general SF problems, using BFS-style expansion with adaptive chunk sizing to mitigate device-memory exhaustion; however, GPU-only execution remains vulnerable to deep and irregular search trees, and failed chunks may waste computation. Existing CPU-GPU graph-processing systems [14, 28, 31, 42] mainly target iterative, vertex-centric, or approximate workloads rather than exact recursive SF. G-ThinkerCG instead provides a unified backtracking-oriented execution model for SF that combines memory-bounded GPU exploration with CPU processing of deep and irregular tasks.

3 BFS-DFS SUBGRAPH EXTENSION ON GPU

GPU-based SF must balance two conflicting goals. Level-wise BFS subgraph extension exposes abundant parallelism for efficient SIMD execution, but may generate an excessively large frontier and exhaust device memory. DFS avoids materializing large frontiers, but provides less parallelism and requires per-warp stacks whose sizes depend on the search depth. The challenge here is that the subgraph enumeration tree is wide and the depth of different branches can vary significantly and may not be known in advance.

G-ThinkerCG addresses this tension with a memory-bounded hybrid BFS-DFS strategy. It performs parallel level-wise extension while bounding the number of subgraphs materialized at each level. When this bound is reached, the GPU temporarily suspends the remaining work at the current level, explores the generated subgraphs at the next level, and later backtracks to the suspended work. In addition, when a deep search path exhausts the device buffer despite bounded per-level expansion, generated subgraphs are temporarily spilled to host memory and resumed later. Together, these mechanisms retain the parallelism of BFS while allowing the search to progress to arbitrary depths with bounded device memory.

This section focuses on this GPU-side execution mechanism in isolation; Section 4 explains how it is integrated into the complete G-ThinkerCG system with CPU-GPU co-processing.

3.1 Hybrid BFS-DFS Extension

A *GPU worker*, implemented as a host CPU thread, drives GPU execution by repeatedly launching kernels over batches of subgraphs for computations and extensions. Logically, subgraphs are organized by enumeration level in a device buffer $\mathcal{B}$. Given the subgraphs at current level L_i , GPU warps process them in parallel and generate their valid extensions at the next level L_{i+1} .

Applications express GPU-side extension through two device user-defined functions (UDFs). The optional UDF `process(.)` computes candidate elements (vertices or edges) for extending a segment of parent subgraphs in L_i , while the other UDF `extend(.)` materializes the corresponding child subgraphs in L_{i+1} using these

candidates. Separating candidate generation from materialization enables work to be distributed at candidate granularity when candidate counts for parent subgraphs are skewed. Applications may alternatively generate candidates and extensions directly in `extend(.)`, avoiding an intermediate candidate buffer. Both methods use the scheduling and memory-management mechanism described below.

Bounded Level-Wise Extension. Pure BFS repeatedly materializes the entire next level before proceeding, so a wide enumeration tree may quickly exhaust GPU memory. Instead, G-ThinkerCG limits each round of level-wise expansion to η newly generated subgraphs, where η is chosen sufficiently large to expose substantially more parallel work than the number of GPU warps. Subgraphs at each level are stored contiguously in the device buffer $\mathcal{B}$; for illustration, each subgraph is treated as one logical position. See Appendix C for the actual variable-length representation.

Figure 2 illustrates the execution on two initial subgraph-tasks T_A and T_B , which form the initial task chunk at level L_1 . Here, A extends to CDE and B to FGH at L_2 , and each L_2 subgraph further generates three children at L_3 ; we set the expansion bound to $\eta = 6$.

The GPU first performs level-wise expansion as in BFS: extending A and B generates the six subgraphs C–H at L_2 . Since the bound η is reached, we then begin expanding L_2 . After C and D generate I–N, the L_3 frontier reaches η , so instead of continuing to expand the remaining parents E–H, we record their position range [4, 8] on a stack S_L (for later backtracking) and immediately process L_3 .

Once the current deeper frontier is exhausted, the GPU pops the most recently deferred range from S_L and resumes its expansion. For example, it next extends E and F, generating O–T and leaving G–H as another deferred range [6, 8]. Repeating this process interleaves breadth-wise parallel expansion with depth-wise backtracking, bounding the number of newly materialized subgraphs at each step while retaining substantial GPU parallelism.

Implementation. The GPU worker implements the above procedure by repeatedly invoking the application UDFs on the current level. Before each kernel invocation, the runtime automatically sets the corresponding read and write segments in $\mathcal{B}$, so that the UDF reads parent subgraphs from L_i and appends newly generated subgraphs to L_{i+1} . Within a kernel, GPU warps acquire parent subgraphs and reserve output positions through system-provided atomic buffer primitives, detailed in Appendix C which introduces the subgraph buffer data structure. Thus, user code only specifies how each subgraph is processed and extended, while level management, work distribution, and buffer positioning are handled by the runtime, the control flow of which is detailed in Appendix B.

The runtime also maintains lightweight counters that allow the UDF kernels to stop acquiring new parents once the number of

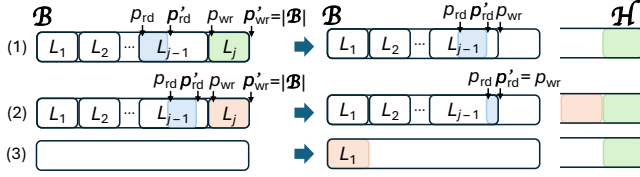


Figure 3: Example of Host Buffering of Subgraphs

generated subgraphs reaches η . Since multiple warps execute concurrently, some warps may already be extending their assigned parents when the threshold is reached, so the resulting frontier can slightly exceed η ; hence, η acts as a soft bound on per-round expansion rather than a strict capacity limit. This does not compromise memory safety: when the device buffer $\mathcal{B}$ itself becomes full, newly generated subgraphs that cannot be accommodated are spilled to a host-side buffer and resumed later, as described next.

3.2 Host Buffering for Deep Search

Bounding each generated frontier prevents width-induced memory explosion, but a sufficiently deep search can still exhaust device memory because deferred work from multiple levels may coexist in $\mathcal{B}$. Thus, the runtime must also externalize accumulated work when the device buffer becomes full.

To this end, G-ThinkerCG augments the device buffer $\mathcal{B}$ with a host-resident spill stack $\mathcal{H}$, allocated in CUDA managed memory so that it is directly accessible by the GPU, as illustrated in Figure 3.

During extension, the runtime monitors the available capacity of $\mathcal{B}$ as warps append newly generated subgraphs. If an append would exceed the device-buffer capacity, the generated subgraph is redirected to $\mathcal{H}$ rather than discarded, and the GPU stops acquiring additional parents for the current expansion. After the `extend(.)` kernel completes, the generated frontier is deferred to $\mathcal{H}$, allowing the GPU to backtrack and continue processing the remaining work in $\mathcal{B}$ (see Figure 3(1)–(2)). Once the current device-resident batch is exhausted, the GPU worker first checks $\mathcal{H}$: if it is non-empty, up to η spilled subgraphs are popped into $\mathcal{B}$ as a new input batch and processed by the same hybrid BFS-DFS procedure (see Figure 3(3)); otherwise, the worker proceeds to the next regular input chunk.

Using a stack for $\mathcal{H}$ preserves a near-DFS traversal order, allowing recently spilled deep tasks to resume first and helping limit host-memory growth in practice.

In this design, spilling does not waste any computation: each spilled subgraph represents an unexplored subtree of the enumeration tree and can be resumed independently when device space becomes available. This contrasts with the overflow handling of G^2 -AIMD [43] that retries failed BFS chunks after size adjustment, where work already performed for an overflowing chunk is lost.

The device buffer $\mathcal{B}$ and host stack $\mathcal{H}$ share a common compact representation for variable-length subgraphs, enabling subgraphs to move between them without application-specific conversion. Appendix C describes the buffer representation and the warp-safe primitives for subgraph appending and fetching in detail.

Together, the two mechanisms address complementary sources of memory pressure: the expansion threshold η controls frontier growth caused by wide enumeration levels, while host spilling allows the search to continue when accumulated work from deep enumeration paths exceeds device capacity.

4 SYSTEM DESIGN

This section presents the overall computation model and system design of G-ThinkerCG. Building on the GPU-side hybrid BFS-DFS execution mechanisms described in Section 3, we show how recursive CPU search and parallel GPU exploration are integrated through a common subgraph-task abstraction.

The key idea is to use degeneracy ordering as a heuristic to initially steer the many shallow subgraph-tasks toward the GPU for massively parallel batched execution, while directing the few straggler-prone tasks with deep enumeration trees toward CPUs, and then dynamically correct workload imbalance through task decomposition and CPU-GPU work stealing.

Overview. Figure 4 gives a high-level overview of the G-ThinkerCG architecture, which runs on a single multi-core machine equipped with a GPU. A G-ThinkerCG program consists of a master thread, a group of CPU workers, and a GPU worker.

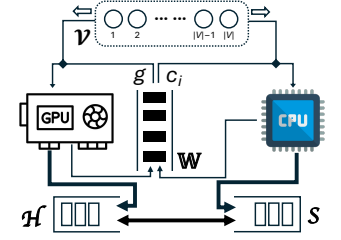


Figure 4: System Overview

The master maintains global work pools and assigns batches of work to idle workers, while workers process their assigned batches independently. CPU workers execute subgraph-tasks using recursive, task-centric DFS to handle deep and irregular enumeration trees, whereas the GPU worker processes subgraph-tasks in parallel using the hybrid BFS-DFS strategy described in Section 3.

The common scheduling unit is the subgraph-task defined in Section 2.1: a task T_S represents all search work in the enumeration subtree rooted at subgraph S . CPU and GPU workers process such tasks differently (using recursive DFS on CPUs and parallel BFS-DFS on the GPU) but either can continue the search represented by a task. This common task abstraction allows subgraph-tasks to be exchanged between CPU and GPU through host-resident work pools for dynamic load balancing.

4.1 Heterogeneous Task Scheduling

Work Organization. Algorithm 2 provides a pseudocode summary of the master thread, which schedules work from an ordered queue $\mathcal{V}$ of initial vertices and a stack $\mathcal{S}$ of dynamically generated subgraph-tasks. Each vertex $v \in \mathcal{V}$ represents an initial task rooted at $\{v\}$, whereas $\mathcal{S}$ stores tasks produced when CPU workers decompose long-running tasks, making their remaining search subtrees available for redistribution; we describe this decomposition shortly. Idle CPU and GPU workers are maintained in a worker queue $\mathcal{W}$.

Each worker receives work through two local buffers: $\mathcal{L}_v$ for a batch of vertices from $\mathcal{V}$ and $\mathcal{L}_t$ for a batch of tasks from $\mathcal{S}$. The master dispatches work only through these buffers, after which the worker processes the assigned batch independently.

Initial CPU-GPU Scheduling. G-ThinkerCG allows users to order the vertices in $\mathcal{V}$ using degeneracy ordering, obtained through the peeling algorithm for k -core decomposition [5]. This ordering tends to place vertices with smaller candidate sets earlier and is widely adopted in set-enumeration-based subgraph finding algorithms [4, 12]. As illustrated in Figure 1, the vertices under the root can be

Algorithm 2 Pseudocode of the Master Thread

```

1: Create  $n_c$  CPU workers and a GPU worker and add them to  $\mathbb{W}$ 
2:  $\mathcal{S} \leftarrow \emptyset, \mathcal{V} \leftarrow V$  // note that  $G = (V, E)$ 
3: while  $|\mathbb{W}| \neq n_c + 1$  or  $\mathcal{V} \neq \emptyset$  or  $\mathcal{S} \neq \emptyset$  do
4:   if  $\mathbb{W} = \emptyset$  then wait to be notified by a worker
5:   if  $\mathcal{V} = \emptyset$  and  $\mathcal{S} = \emptyset$  then continue
6:    $w \leftarrow \mathbb{W}.\text{DEQUEUE}()$ 
7:   if  $w$  is a GPU worker then
8:     if  $\mathcal{V} \neq \emptyset$  then  $w.\mathcal{L}_v \leftarrow \mathcal{V}.\text{POP\_FRONT\_BATCH}()$ 
9:     else  $w.\mathcal{L}_t \leftarrow \mathcal{S}.\text{POP\_BATCH}()$ 
10:  else //  $w$  is a CPU worker
11:    if  $\mathcal{S} \neq \emptyset$  then  $w.\mathcal{L}_t \leftarrow \mathcal{S}.\text{POP\_BATCH}()$ 
12:    else  $w.\mathcal{L}_v \leftarrow \mathcal{V}.\text{POP\_BACK\_BATCH}()$ 
13:    notify  $w$  to wake up for processing
14:  $\text{global\_end\_flag} \leftarrow \text{true}$ , notify all workers (to terminate)

```

ordered as $\{d\}, \{c\}, \{b\}, \{a\}$, so that the subtrees near the front tend to be shallower while those near the back tend to be deeper.

The master exploits this ordering to provide an initial CPU-GPU scheduling bias. When the GPU worker becomes idle and $\mathcal{V}$ is non-empty, it receives a large vertex chunk from the *front* of $\mathcal{V}$ (Line 8). Such a chunk typically contains many initial tasks with relatively shallow enumeration trees; processing them together provides abundant parallel work for the GPU. In contrast, when a CPU worker requests new work but no dynamically generated task is available in $\mathcal{S}$ (Line 12), it receives a small vertex chunk from the *back* of $\mathcal{V}$, where the few straggler-prone tasks with deep or irregular enumeration trees are more likely to occur.

Degeneracy ordering is only an initial heuristic rather than a rigid partition of the search space. The actual depth and cost of an enumeration subtree are not known until it is explored, so G-ThinkerCG further relies on dynamic task redistribution to correct imbalanced assignments. This is repetitive, also with degeneracy it is expected that CPU will get those large tasks

Dynamic Task Decomposition and Redistribution. ~~Degeneracy ordering only provides an initial scheduling heuristic, and a CPU worker may still encounter a task whose enumeration subtree is unexpectedly large.~~ To prevent such a task from becoming a straggler, G-ThinkerCG adopts the timeout-based task decomposition mechanism from prior G-thinker-style systems [20, 40, 44]. If a subgraph-task T_S runs longer than a threshold τ_{time} , the CPU worker stops exploring the task as a single unit and exposes its remaining unexplored branches as smaller subgraph-tasks. Each resulting task is rooted at a subgraph reached during the search and represents an independent enumeration subtree, so it can subsequently be processed by another worker without repeating the portion of T_S that has already been explored.

The newly generated tasks are first buffered locally and then inserted in batches into the global stack $\mathcal{S}$, where they become available for redistribution. Batching reduces synchronization on the shared stack, while placing the tasks in $\mathcal{S}$ prevents a single CPU worker from being tied to a long-running subtree. CPU workers give priority to tasks in $\mathcal{S}$ over new vertices from $\mathcal{V}$, so decomposed work is consumed promptly (Lines 11–12). Once the GPU has exhausted the initial vertices in $\mathcal{V}$, it can also obtain a batch of tasks from $\mathcal{S}$ (Line 9), allowing CPU-exposed parallel work to be stolen and processed by the GPU.

Conversely, the GPU may accumulate spilled subgraph-tasks in its host-side stack $\mathcal{H}$ during the execution described in Section 3.2. When the number of idle CPU workers exceeds a threshold τ_c , the GPU worker returns a batch of tasks from $\mathcal{H}$ to $\mathcal{S}$, making them available for CPU processing. The threshold τ_c balances timely utilization of idle CPUs against excessive task migration between the GPU and CPUs. Thus, G-ThinkerCG supports bidirectional work redistribution between heterogeneous processors via $\mathcal{S}$.

Overall, degeneracy ordering provides an inexpensive initial bias, while timeout-based decomposition and bidirectional task exchange dynamically correct the assignment as the actual search structure becomes known. This allows abundant shallow work to remain on the GPU while deep or straggler-prone work is preferentially absorbed by CPU workers.

The master detects global termination when both $\mathcal{V}$ and $\mathcal{S}$ are empty and all CPU and GPU workers are idle, waiting in $\mathbb{W}$. Since no unassigned or dynamically generated subgraph-task remains at this point, the master notifies all workers to terminate.

4.2 GPU and CPU Workers

GPU Worker. The GPU worker is a dedicated host-side CPU thread responsible for converting assigned work into GPU input batches and launching GPU kernels. It may receive either initial vertices through $\mathcal{L}_v$ or subgraph-tasks through $\mathcal{L}_t$ (recall Lines 7–9 of Algorithm 2). Tasks received through $\mathcal{L}_t$ are transferred into the host spill stack $\mathcal{H}$, from which the worker schedules batches for GPU processing. The worker prioritizes tasks already in $\mathcal{H}$, which preserves a near-DFS execution order and prevents pending tasks in $\mathcal{H}$ from accumulating; only when $\mathcal{H}$ is empty does it generate initial subgraphs from $\mathcal{L}_v$. Each batch is then processed using the hybrid BFS-DFS execution mechanism of Section 3.

The GPU worker also serves as the handoff point for GPU-to-CPU task migration. When the number of idle CPU workers exceeds τ_c , it may return a batch of tasks from $\mathcal{H}$ to $\mathcal{S}$ for CPU processing. The master does not directly manipulate the GPU execution buffer $\mathcal{B}$ or the host spill stack $\mathcal{H}$; it only mediates task delivery between $\mathcal{S}$ and $\mathcal{L}_t$. This separation allows the GPU worker to manage its execution state without fine-grained synchronization with the master. Appendix D details the complete control flow of the GPU worker.

CPU Workers. Each CPU worker is a host-side thread that processes subgraph-tasks using recursive DFS. As specified in Algorithm 2, the master gives CPU workers priority to decomposed tasks in $\mathcal{S}$, which are delivered through $\mathcal{L}_t$; only when $\mathcal{S}$ is empty does it assign initial vertices through $\mathcal{L}_v$. For tasks received through $\mathcal{L}_t$, the CPU worker directly explores their enumeration subtrees using application-specific recursive backtracking. For vertices received through $\mathcal{L}_v$, it first creates the corresponding root tasks and then processes them in the same manner. This task-centric execution closely follows the corresponding sequential SF algorithm [24, 44] and is well suited to deep search spaces with irregular control flow that are less amenable to level-wise GPU execution.

To prevent a deep or unexpectedly expensive task from monopolizing a CPU worker, G-ThinkerCG applies the timeout-based decomposition described in Section 4.1. If a task runs longer than τ_{time} , the worker decomposes its remaining search space into smaller subgraph-tasks and returns them to $\mathcal{S}$ for redistribution. Newly

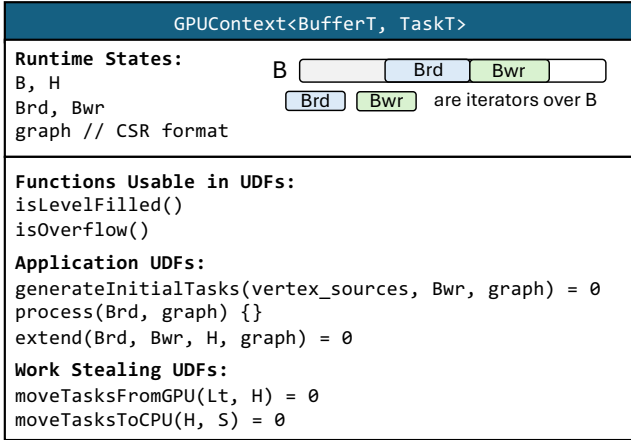


Figure 5: Simplified GPUContext class with UDFs

generated tasks are buffered locally at the CPU worker and flushed to S in batches, reducing contention on the shared stack.

Although CPU and GPU workers use fundamentally different execution strategies (recursive DFS on CPUs and bounded parallel BFS-DFS on the GPU), both operate on the same logical subgraph-task abstraction. This common task boundary allows work to migrate between CPU and GPU while letting each processor execute it using the strategy best suited to its hardware and workload characteristics.

5 PROGRAMMING INTERFACE

G-ThinkerCG is designed to make existing subgraph-centric applications easy to port while supporting the new heterogeneous execution model described above. Rather than introducing an entirely new application interface, we build on programming abstractions already familiar to users of prior SF systems: the GPU interface follows the subgraph-centric programming style of G^2 -AIMD [43], while the CPU interface follows the recursive task-centric model of G-thinker and G-thinkerQ [40, 44]. This allows existing GPU and CPU implementations to be adapted to G-ThinkerCG without restructuring their core application-specific search logic.

Reusing familiar application-facing abstractions is orthogonal to the new execution mechanisms of G-ThinkerCG. We extend the GPU interface of G^2 -AIMD [43] (which only adopts `extend()`) with the optional `process()` UDF for finer-grained load balancing, and, more importantly, unify CPU and GPU execution through the common subgraph-task abstraction of Section 2.1. The runtime can, therefore, migrate logical subgraph-tasks between CPU and GPU while allowing each processor to retain its natural execution model. This interface is coupled with the hybrid BFS-DFS execution and host buffering of Section 3, as well as the degeneracy-based scheduling and dynamic task redistribution of Section 4.1.

5.1 GPU Programming Interface

Figure 5 summarizes the application-facing GPU interface. A GPU application specializes `GPUContext<BufferT, TaskT>`, where `BufferT` specifies the GPU-side representation of subgraphs in $\mathcal{B}$ and $\mathcal{H}$. It can directly use `BufferBase` from Appendix C, or extend it with application-specific subgraph state. The other template parameter, `TaskT`, represents a subgraph-task on the CPU side and is used for

CPU-GPU task exchange, as detailed in Section 5.2. The context provides the runtime states and functions needed by application UDFs. As illustrated in the figure, `Brd` and `Bwr` are read and write views of type `BufferT` over the current input and output levels in $\mathcal{B}$, respectively. The runtime manages these states, while applications primarily implement three device UDFs as follows to specify subgraph generation and extension.

The UDF `generateInitialTasks()` generates initial subgraphs into `Bwr` from the vertex batch assigned through $\mathcal{L}_v$ from $\mathcal{V}$. The optional UDF `process()` computes candidate elements for parent subgraphs acquired from `Brd`, while UDF `extend()` materializes child subgraphs and appends them to `Bwr` (or $\mathcal{H}$ upon device-buffer overflow). The latter two UDFs constitute the application-specific extension logic used by the hybrid BFS-DFS execution of Section 3. Applications may alternatively omit `process()` and perform both candidate generation and subgraph materialization directly in `extend()`.

The GPU interface follows the subgraph-centric programming model of G^2 -AIMD [43] (which exposes UDFs `generateInitialTasks()` and `extend()`), but introduces the optional `process()` UDF for finer-grained load balancing. In G^2 -AIMD, a warp that acquires a parent subgraph is responsible for extending it with all of its candidates, which can cause severe imbalance when candidate counts are highly skewed. With `process()`, candidate generation is separated from materialization: candidates from different parents are first collected in an intermediate buffer Q and can then be distributed more evenly during `extend()`. The simpler one-stage interface remains available when such separation is unnecessary.

In addition to the application UDFs, the runtime exposes two lightweight predicates that UDFs can query during GPU execution. `isLevelFilled()` tells a UDF whether to stop acquiring new parent subgraphs for the current bounded expansion, while `isOverflow()` reports whether the device buffer is approaching capacity, allowing newly generated subgraphs to be redirected to $\mathcal{H}$. UDFs fetch and append subgraphs through the `BufferT` primitives described in Appendix C. Together, these interfaces let application developers focus on subgraph generation, pruning, and extension, while the runtime transparently manages work distribution, hybrid BFS-DFS backtracking, and memory-bounded execution. The complete `GPUContext` interface and its interaction with the GPU-worker control flow are detailed in Appendix D.

5.2 Unified CPU-GPU Programming Interface

Figure 6 shows how the GPU interface above is integrated with the task-centric CPU programming model. Classes shown in orange are user-defined and specify application-specific logic, while the remaining classes are provided by the runtime. Rather than forcing CPU and GPU execution into a common low-level interface, G-ThinkerCG preserves the programming style natural to each processor and unifies them at the logical subgraph-task level.

On the CPU side, G-ThinkerCG follows the task-based programming model of G-thinker and G-thinkerQ [40, 44]. An application defines a context type `ContextT` that stores the state required to explore a subgraph-task, which is wrapped by `Task<ContextT>` to form the application-specific task type `TaskT`. Users implement `taskSpawn(v)` to create an initial task from vertex v and `compute(c)` to recursively explore and extend the task’s subgraph represented

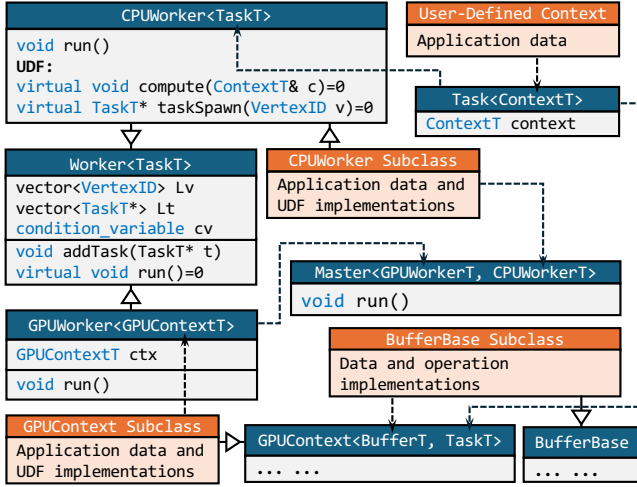


Figure 6: C++ Programming Interface of G-thinkerCG

by context c . The resulting CPU code therefore remains close to the corresponding sequential backtracking algorithm.

For example, consider the Bron-Kerbosch algorithm in Algorithm 1. Its task context can directly maintain the three sets (R, P, X) . The UDF $\text{taskSpawn}(v)$ creates the initial state for the enumeration subtree rooted at $\{v\}$, while $\text{compute}(c)$ implements the recursive Bron-Kerbosch search over the (R, P, X) state stored in c . The runtime augments this familiar recursive execution with the timeout-based task decomposition described in Section 4.1 (see Section 6), allowing a long-running recursive search to expose its remaining branches as independent subgraph-tasks for redistribution.

CPU and GPU implementations may use different physical representations for these tasks. On the CPU, a task T_S is represented by a TaskT carrying its application context; on the GPU, the same logical task is represented compactly using BufferT for parallel subgraph extension. What is shared is not the physical data structure but the semantics of T_S : both representations describe the unexplored enumeration subtree rooted at subgraph S . This common abstraction is what allows work to migrate between the two execution models.

To cross this representation boundary, the GPU interface GPUContext (recall Figure 5) also provides two host UDFs. $\text{moveTasksFromCPU}(\cdot)$ converts CPU-side TaskT objects assigned through $\mathcal{L}_t$ into the BufferT representation used in $\mathcal{H}$, while $\text{moveTasksToCPU}(\cdot)$ performs the reverse conversion when GPU-side tasks are returned to S for CPU processing. These conversion UDFs implement the application-specific state translation required by the bidirectional task redistribution of Section 4.1; Appendix D describes their invocation by the GPU worker in detail.

At the system level, CPUWorker and GPUWorker share the base Worker abstraction, which provides the local input buffers $\mathcal{L}_o$ and $\mathcal{L}_t$ used by the master for scheduling. An application specializes the CPU side with its $\text{ContextT}/\text{TaskT}$ logic (via UDFs of CPUWorker) and the GPU side with its $\text{BufferT}/\text{GPUContext}$ logic (via UDFs of GPUContext). The resulting worker types are managed by the Master, whose scheduling logic is independent of the application-specific search implementation. Note that we cannot directly move

the UDFs of GPUContext to GPUWorker and let its subclass implement these UDFs to pass to Master, since CUDA does not support virtual functions or polymorphism in device code.

Overall, G-ThinkerCG preserves familiar CPU and GPU subgraph-centric programming styles while providing a new common boundary, i.e., the subgraph-task, at which heterogeneous scheduling and work migration occur. Existing application logic can, therefore, retain the execution style for which it was designed, while the runtime exploits CPU and GPU jointly through memory-bounded GPU execution and dynamic heterogeneous scheduling.

6 APPLICATIONS

We implement three representative SF applications on G-ThinkerCG: maximal clique enumeration (MCE), maximal quasi-clique mining (MQC), and subgraph matching (SM). We select these applications because mature subgraph-centric implementations already exist for both GPU and CPU execution, allowing us to largely preserve their application-specific search and pruning logic and focus on evaluating G-ThinkerCG’s new execution and scheduling mechanisms.

On the GPU side, the MCE and SM implementations are ported from $G^2\text{-AIMD}$ [43], with their core subgraph-extension logic in $\text{extend}(\cdot)$ retained. For MQC, we adapt the subgraph-extension logic of a recent GPU implementation cuQC [19] to G-ThinkerCG; due to [19]’s poor GPU buffer management (to be explain when we describe the MQC application), it returns incorrect results or fails on most graphs we tested, which is resolved by adapting the subgraph-extension logic to execute with our G-ThinkerCG framework.

On the CPU side, MQC and SM build directly on their existing G-thinkerQ implementations [44], while MCE is implemented using the same task-centric programming interface. All three use the same CPUWorker -level UDF structure and timeout-based task decomposition mechanism described in Section 5.

Workload Characteristics. MCE and MQC exhibit the irregular search behavior that motivates CPU-GPU co-processing. After initial vertices are arranged in degeneracy order, most vertices tend to induce relatively shallow enumeration trees, whereas a small number of vertices toward the back of the ordering can induce exceptionally deep and expensive trees. Neither CPU-only nor GPU-only execution is ideal for such highly skewed workloads. CPU-only execution processes the large number of shallow trees with only a limited number of CPU cores, providing far less aggregate parallelism than a GPU and therefore taking substantially longer time. Conversely, when GPU execution reaches the few deep stragglers, most other trees in the batch have already completed, leaving insufficient parallel work at deeper levels to occupy the GPU warps and causing substantial resource underutilization.

G-ThinkerCG exploits this asymmetry by processing the abundant shallow work in large GPU batches while allowing deep stragglers to be handled by CPU workers using recursive DFS. This division of labor yields a super-additive benefit rather than merely combining CPU and GPU throughput: CPU-only execution is dominated by the massive number of shallow tasks, while GPU-only execution can be dominated by the few deep stragglers. By assigning each workload regime to the processor that handles it efficiently, hybrid execution avoids both bottlenecks and can therefore achieve speedups well beyond $2\times$ over either processor alone.

In contrast, SM has a fundamentally different search structure. Its enumeration depth is bounded by the number of vertices in the query graph, so tasks have the same small maximum depth regardless of their initial data vertex. The large collection of bounded-depth tasks is therefore naturally suited to massively parallel GPU execution, and assigning a portion of them to the slower CPU execution path can instead delay overall completion. As we shall see in our experiments on SM, the GPU-only execution mode of G-ThinkerCG is more efficient than hybrid CPU-GPU execution.

GPU Memory Organization. All three applications follow similar GPU memory organization. Device memory is primarily divided among three components: the input graph, stored in CSR format; the subgraph execution buffer $\mathcal{B}$ managed by G-ThinkerCG; and application-specific temporary workspace. In particular, each GPU warp is assigned a private workspace for intermediate set operations, such as intersections used to compute candidate vertices. These per-warp regions avoid interference among concurrently processed subgraphs and are reused throughout execution.

Applications specialize BufferT to encode the state carried by each GPU subgraph-task. As described in Appendix C, the underlying BufferBase stores variable-length subgraphs as contiguous segments in `vertices[]`, with `offsets[]` identifying the segment belonging to each subgraph. An application may serialize multiple logical components (e.g., the vertices already selected into the subgraph and the candidates available for further extension) within the same segment, together with the metadata needed to recover their boundaries. When additional per-element state is required (e.g., for checking degree-based pruning rules), BufferT can extend BufferBase with auxiliary arrays aligned with `vertices[]`.

Application 1: MCE. For MCE, we adopt the pivoting variant of the Bron-Kerbosch (BK) algorithm used by G^2 -AIMD [43]. Each search state maintains $\langle R, P, X \rangle$, where R is the current clique, P contains candidate vertices for extension, and X records candidates already explored in sibling branches. The pivoting variant selects a pivot from $P \cup X$ and branches only on eligible vertices outside the pivot’s neighborhood, reducing redundant recursive branches. On the GPU, the main operations are set intersections over these state sets and graph adjacency lists. A warp cooperatively performs the required intersections for pivot selection and for constructing the child state after extending with a candidate v , including operations such as $P \cap N(v)$ and $X \cap N(v)$; the intersection results are stored in the warp-private workspace described above.

The GPU representation extends BufferBase with an auxiliary `labels[]` array. The three sets $\langle R, P, X \rangle$ are flattened into a single variable-length segment in `vertices[]`, while the position-aligned entry in `labels[]` records whether each stored vertex belongs to R , P , or X . Thus, a BK state remains a single variable-length subgraph segment addressed by `offsets[]`, while the application-specific labels are carried alongside the vertex data without changing the runtime’s buffer-management operations.

On the CPU, we implement BK with pivoting following the task-centric programming model of G-thinkerQ [24, 44]. Each task context stores a BK state $\langle R, P, X \rangle$. Given an initial vertex v , the CPUWorker UDF `taskSpawn(v)` creates the state $\langle \{v\}, \{u \in N(v) \mid u > v\}, \{u \in N(v) \mid u < v\} \rangle$, where the vertex order follows the degeneracy ordering used for initial task scheduling. The UDF

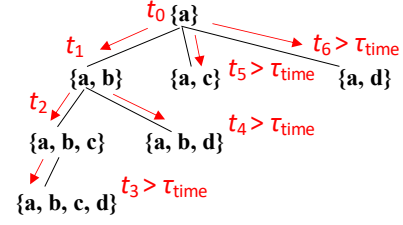


Figure 7: Timeout Example

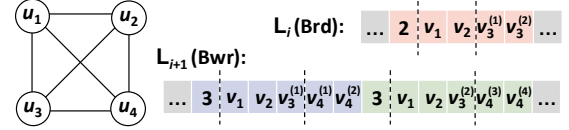


Figure 8: Illustration of Super-Subgraph Scheme

compute(c) then performs recursive BK backtracking with pivoting on the $\langle R, P, X \rangle$ state stored in context c .

To handle stragglers, each CPU task records its creation time t_0 and periodically checks whether its execution time exceeds τ_{time} . Using the basic BK procedure in Algorithm 1 as an example, before recursively processing a child at Line 5, compute(c) checks whether $t_{\text{cur}} - t_0 > \tau_{\text{time}}$. If so, instead of recursively exploring the child state $\langle R \cup \{v\}, P \cap N(v), X \cap N(v) \rangle$, the worker materializes it as a new subgraph-task and calls `addTask(.)` to make it available for redistribution. As illustrated in Figure 7, when the task rooted at $\{a\}$ times out, the unexplored subtrees rooted at $\{a, b, c, d\}$, $\{a, b, d\}$, $\{a, c\}$, and $\{a, d\}$ are progressively exposed as independent tasks during backtracking. The runtime first accumulates these tasks in a local buffer $\mathcal{L}_o$ and then flushes them in batches to the global stack $\mathcal{S}$, reducing synchronization overhead.

Application 2: Subgraph Matching. For SM, the CPU implementation directly reuses G-thinkerQ [44], so we focus on its GPU implementation. We follow the super-subgraph scheme (aka. prefix mode) of G^2 -AIMD [43], illustrated in Figure 8. Rather than materializing one task for every candidate extension, a super-subgraph groups a partial match with all candidates for matching the next query vertex. This grouping allows candidate extensions sharing the same partial match to reuse intermediate computation, particularly the set intersections used to derive their candidate sets.

To illustrate, Figure 8 considers 4-clique matching. The super-subgraph in L_i contains the current partial match $\langle v_1, v_2 \rangle$ (to query vertices $\langle u_1, u_2 \rangle$) followed by two candidates $v_3^{(1)}$ and $v_3^{(2)}$ to match u_3 . Both candidates share the intersection $N(v_1) \cap N(v_2)$, which is computed once for the common prefix rather than independently for the two extensions. Extending these two candidates produces two super-subgraphs in L_{i+1} , each sharing its matched prefix among the candidates for the next query vertex. For example, when a warp extends $\langle v_1, v_2 \rangle$ with $v_3^{(1)}$, we obtain a new super-subgraph in L_{i+1} containing the partial match $\langle v_1, v_2, v_3^{(1)} \rangle$ followed by candidates $N(v_1) \cap N(v_2) \cap N(v_3^{(1)}) = \{v_4^{(1)}, v_4^{(2)}\}$. The candidate set for the next query vertex u is computed by intersecting the neighborhoods of the already matched data vertices whose query vertices are neighbors of u . Keeping these candidates together with their common prefix avoids repeatedly reconstructing the prefix state and recomputing the intersections shared by its extensions.

We slightly modify the physical representation of a super-subgraph to preserve G-ThinkerCG’s generic BufferBase interface. G²-AIMD stores an additional middle offset m in each super-subgraph descriptor, using (s, m, e) to separate the matched prefix vertices $[s, m)$ from its candidates vertices $[m, e)$. In G-ThinkerCG, offsets[] retains the standard (s, e) descriptor of BufferBase; instead, the prefix length is instead serialized as the first entry of the corresponding vertices[] segment. For example, the values 2 and 3 in Figure 8 indicate the lengths of the matched prefixes in L_i and L_{i+1} , respectively. This keeps the runtime buffer representation application-independent and allows SM to specialize the subgraph representation through a BufferT subclass without modifying BufferBase.

The prefix mode reduces redundant computation through such sharing, but grouping all candidates of a prefix into one super-subgraph can create GPU stragglers when candidate-set sizes are highly skewed, because the entire candidate set is processed by the same warp. We, therefore, bound each super-subgraph to at most 100 candidates. If an extension produces a larger candidate set, it is partitioned into multiple super-subgraphs that replicate the same matched prefix but contain disjoint candidate chunks. For example, a set of 250 candidates is materialized as three super-subgraphs containing 100, 100, and 50 candidates, respectively. These super-subgraphs can subsequently be acquired by different warps, exposing additional parallelism while preserving computation sharing within each candidate chunk. This bounded grouping therefore balances the reuse benefit of the super-subgraph scheme against the load imbalance caused by excessively large candidate sets.

Application 3: MQC. Given a density threshold γ , a vertex set S induces a γ -quasi-clique if every vertex in S has at least $\lceil \gamma(|S| - 1) \rceil$ neighbors within S ; it is maximal if no strict superset of S is also a γ -quasi-clique. The MQC query additionally requires $|S| \geq \tau_{\text{size}}$ to exclude small maximal quasi-cliques [20, 44]. A larger τ_{size} also enables more aggressive pruning during enumeration. Thus, an MQC query is parameterized by $(\gamma, \tau_{\text{size}})$, where increasing γ imposes a stronger density requirement and increasing τ_{size} imposes a larger minimum result size. Following prior works on MQC queries, we consider $\gamma \geq 0.5$, for which a γ -quasi-clique has diameter at most two. Following cuQC [19], we precompute and maintain the 2-hop neighborhood of every vertex in addition to the CSR graph, allowing the candidate set of each initial task to be restricted to vertices within two hops of its root.

The enumeration algorithm maintains a selected vertex set S and an extension-candidate set C , and recursively moves candidates from C into S , similar in structure to clique enumeration but with substantially more pruning logic. In particular, the pruning rules repeatedly examine vertex degrees within S and C to determine whether a candidate can participate in any valid extension or whether the entire subtree can be pruned. To support these tests efficiently on the GPU, our BufferT stores S and C in vertices[] and maintains position-aligned auxiliary arrays for application-specific state. For example, two such arrays record each vertex’s degree within S and within C .

The resulting MQC tasks can be much larger than those of MCE or SM, particularly near the roots of the enumeration trees where C may contain a large 2-hop neighborhood. We retain the GPU

subgraph-extension and pruning logic of the recent cuQC implementation [19], but replace its task-buffer management. cuQC assigns each warp a fixed-capacity task buffer: small tasks leave part of the reserved space unused, whereas a task exceeding the per-warp capacity cannot be represented safely. In its current implementation, candidate sets of size at most 70 use shared-memory scratch space, sets of size 71–32,000 use per-warp global-memory scratch space, and larger sets are not processed. Consequently, on most of our tested graphs, the original cuQC implementation either fails or misses results due to such oversized tasks.

G-ThinkerCG instead stores MQC tasks as variable-length segments in the shared execution buffer $\mathcal{B}$. Different warps therefore draw space from the same buffer on demand rather than owning fixed-capacity regions, allowing unused capacity from small tasks to be utilized by larger ones. As the search descends, pruning progressively reduces C , so most generated tasks can be represented and processed efficiently by this common GPU buffer.

Some root tasks can nevertheless be too large for efficient GPU execution even under this shared representation. We detect initial MQC tasks whose candidate sets exceed 32,000 vertices and place them in a dedicated queue Q_{big} for CPU-only processing. CPU workers prioritize these tasks over regular work. As recursive pruning proceeds, their candidate sets typically shrink. When such a CPU task times out and is decomposed, a resulting subtask whose candidate set remains above the threshold is placed in Q_{big} , whereas a sufficiently reduced subtask is inserted into the regular stack $\mathcal{S}$ and can subsequently be scheduled on either CPU or GPU. Thus, oversized tasks initially unsuitable for GPU execution (due to limited device memory) can naturally re-enter heterogeneous scheduling once pruning reduces their memory footprint.

7 EXPERIMENTS

Our evaluation focuses on two complementary comparisons. First, we evaluate G-ThinkerCG under three execution modes: **CPU-Only**, which executes all subgraph-tasks using CPU workers; **GPU-Only**, which executes them using the GPU worker; and **Hybrid**, which enables CPU-GPU co-processing and dynamic task redistribution. This comparison isolates the benefit of heterogeneous execution under different search-space characteristics.

Second, we compare G-ThinkerCG with existing CPU and GPU solutions. Our primary GPU baseline is G²-AIMD [43], while the primary CPU baselines use the same underlying application algorithms as G-ThinkerCG whenever possible. These algorithm-matched comparisons are our main basis for assessing system-level improvements, since they minimize performance differences caused by application-specific algorithmic choices. We additionally report other state-of-the-art implementations where appropriate for broader reference. Our system and application code is publicly available at <https://github.com/akhlaqueak/G-ThinkerCG>.

Experimental Setup. We conducted experiments on a machine with an NVIDIA Tesla A100 GPU (80 GB memory), an AMD EPYC 7763 Milan CPU (64 cores), and 512 GB DDR4 RAM. For **CPU-Only**, we use 32 CPU workers. For **GPU-Only**, the CPU workers are disabled and all subgraph-tasks are processed by the GPU worker. For **Hybrid**, we use the GPU worker together with 32 CPU workers, allowing both processors to execute and exchange subgraph-tasks. GPU kernels use 108 thread blocks with 1024 threads per block.

The input graph is stored in CSR format and copied to device memory before GPU execution. As described in Section 6, device memory is shared by the input graph, application-specific temporary workspace, and the subgraph execution buffer $\mathcal{B}$. After allocating the graph and application-specific workspace, including the per-warp scratch space used for intermediate set operations, we assign the remaining available device memory to $\mathcal{B}$. On the host, the spill stack $\mathcal{H}$ and global task stack $\mathcal{S}$ are each provisioned with half of the available host memory.

After completing an assigned chunk, a worker returns to the idle queue $\mathbb{W}$ and waits for the next scheduling decision (see Figure 4). We generally use much smaller vertex chunks for CPU workers than for the GPU worker, reflecting their different roles in heterogeneous execution. Large GPU chunks aggregate many initial tasks to expose sufficient parallelism and amortize kernel-launch and scheduling overhead, whereas small CPU chunks avoid committing CPU workers to excessive initial work and allow them to respond promptly to dynamically exposed stragglers in $\mathcal{S}$. The appropriate chunk sizes are application-dependent because the relative CPU/GPU throughput and the number of initial tasks differ substantially across SF workloads. We, therefore, configure the chunk sizes separately for MCE, SM, and MQC, as will be described with their respective experiments.

GPU-to-CPU

Unless otherwise specified, we set the CPU timeout threshold to $\tau_{\text{time}} = 0.1$ seconds and the CPU-GPU task-exchange threshold to $\tau_c = n_c/2$, where n_c is the number of CPU workers. The former determines when a long-running CPU task is decomposed into smaller subgraph-tasks for redistribution: a smaller value exposes parallel work more quickly but incurs more frequent task decomposition, whereas a larger value risks leaving stragglers on individual CPU workers for too long. The latter determines when the GPU worker returns tasks from $\mathcal{H}$ to $\mathcal{S}$ for CPU processing (see Appendix D, Algorithm 4, Lines 11–12). Setting $\tau_c = n_c/2$ triggers GPU-to-CPU migration only when a substantial fraction of the CPU workers are idle, balancing timely utilization of available CPU capacity against excessive task migration. We empirically validated these default settings in our parameter study and found that they provide robust performance across the evaluated applications and datasets.

GPU Baseline. Our primary GPU baseline, G^2 -AIMD [43], uses two device buffers that equally partition the available device memory and operate in a ping-pong manner. Starting from an initial task chunk in one buffer, G^2 -AIMD expands all tasks at the current level and writes the generated subgraphs into the other, initially empty buffer. At the next level, the newly generated subgraphs become the input frontier, while the previous input buffer is cleared and reused as the output buffer; the two buffers continue alternating in this manner as the search proceeds. If the generated frontier at any level exceeds the capacity of its output buffer, the execution of that initial chunk is considered failed. G^2 -AIMD then reduces the initial task-chunk size and reruns the chunk from the beginning, discarding the computation already performed in the failed attempt.

The advantage of this design is that, as long as no level overflows, each level can expand its entire input frontier and thus expose substantially more parallel work than G-ThinkerCG’s per-round threshold η permits. G-ThinkerCG makes a different tradeoff. Its

hybrid BFS-DFS execution over the unified buffer $\mathcal{B}$ bounds each expansion using the soft threshold η , but retains completed extensions and continues the remaining search through backtracking or host spilling rather than restarting the initial chunk. We set $\eta = 2,000$ by default, which already exposes substantial GPU parallelism while avoiding the repeated computation caused by failed expansions.

As the following experiments will show, the savings from avoiding such wasted computation generally outweigh the additional level-wise parallelism available to G^2 -AIMD, allowing GPU-Only to achieve better overall performance on most tested workloads.

Performance Comparison on MCE.

..... data source to appendix hybrid
..... chunk size for each application MQC: slide 26 c parameter figure, root task size Table 3 needs update
..... work stealing params?

Performance Comparison on Maximal Clique Enumeration (MCE). Table 1 shows the MCE running time (unit: second) on 20 large real graph datasets for the 3 modes of G-thinkerCG, and our baseline G^2 -AIMD [43]. The Bron-Kerbosch algorithm variant with pivoting is implemented in all systems, and the graphs are all from KONECT [2] and Network Repository [3], selected to be large enough so that MCE is challenging but can be solved in reasonable time. As Table 1 shows, the maximum degree $d_{\max}$ is huge for most graphs, making MCE quite challenging.

We can see that GPU-Only beats G^2 -AIMD on 15 out of the 20 graphs, indicating that our BFS-DFS hybrid strategy is superior than the chunk-adaptive BFS solution of G^2 -AIMD which may incur wasted chunk computation due to OOM.

More importantly, Hybrid is consistently faster than both CPU-only and GPU-only, showing that the CPU-GPU co-processing is very effective. In particular, $\text{Gain}^{\text{H}2\text{G}}$ shows the running time ratio for GPU-only v.s. Hybrid which is up to 9.5 (i.e., Hybrid is up to 9.5× faster than GPU-only), and $\text{Gain}^{\text{H}2\text{C}}$ shows the running time ratio for CPU-only v.s. Hybrid which is up to 3.3.

Performance Comparison on Subgraph Matching (SM). Table 2(c) shows the SM running time (unit: second) on 7 of the 20 graphs used in Table 1 for 3 representative query graphs shown in Table 2(a). The programs run out of our maximum allowed time threshold of 1800 seconds (denoted by OOT) on the other 13 graphs.

Note that while we use the *prefix* mode of [43] by default which lets a warp extend a subgraph g with all its candidates so that the set intersection results on g can be shared, this leads to load balancing problems on *edit-cebwiki* and *edit-shwiki* where many vertices have very high fanouts (see Table 2(b)-**③④**) – unlike the other power-law graphs (see Table 2(b)-**①②**) – which leads to extreme load imbalance. Therefore, we switch to the *expand* mode of [43] (where each warp extends with one candidate) on both graphs for P1 and P2 which reduces the running time significantly. These numbers have been highlighted with asterisks in Table 2(c).

We can see that GPU-Only is faster than G^2 -AIMD in most cases. However, Hybrid does not improve GPU-Only in most cases other than *soc-pokec*. This is because unlike in MCE, all enumeration trees are of the same height (equal to the query subgraph size) in SM, so CPU is not as complementary; tasks assigned for CPU processing lag far behind GPU processing, bottlenecking Hybrid’s

Table 1: Performance Comparison on Maximal Clique Enumeration (MCE)

G	$ V $	$ E $	d_{max}	G ² -AIMD	CPU-Only	GPU-Only	Hybrid	Gain ^{92G}	Gain ^{92C}
<i>soc-sinaweibo</i>	58.65M	522.6M	278K	108.2	205.6	150.1	109.2	1.4	1.9
<i>trackers</i>	27.66M	281.2M	11,571K	51.1	28.8	47.8	30.5	1.6	0.9
<i>wikipedia_link_en</i>	25.92M	1086M	4271K	430.5	334.2	446.4	253.0	1.8	1.3
<i>edit-cebwiki</i>	8.482M	23.58M	8421K	9.9	1.3	7.8	1.2	6.3	1.0
<i>wikipedia_link_ceb</i>	7.891M	127.8M	3636K	17.6	4.5	15.1	2.4	6.2	1.9
<i>edit-svwiki</i>	7.570M	43.88M	5669K	17.4	6.4	14.2	3.8	3.8	1.7
<i>edit-enwiktionary</i>	5.812M	54.23M	3504K	11.5	13.1	10.0	5.8	1.7	2.3
<i>edit-zhwiki</i>	5.005M	33.08M	1208K	6.1	7.8	5.1	3.9	1.3	2.0
<i>edit-itwiki</i>	4.799M	63.84M	1076K	17.8	30.9	15.0	11.8	1.3	2.6
<i>edit-shwiki</i>	4.579M	12.53M	4043K	5.7	1.3	4.3	1.3	3.3	1.0
<i>edit-mgwiktionary</i>	4.063M	17.65M	4059K	0.7	0.3	1.5	0.2	9.5	1.9
<i>soc-livejournal</i>	4.033M	55.86M	2651	5.3	11.6	6.8	4.3	1.6	2.7
<i>edit-viwiki</i>	3.496M	27.30M	2187K	3.0	1.0	2.4	0.8	3.0	1.3
<i>edit-frwiktionary</i>	3.411M	26.85M	2487K	4.5	1.3	3.6	1.2	3.0	1.1
<i>wikipedia_link_sr</i>	3.175M	206.6M	369K	1279.8	725.9	1097.6	462.7	2.4	1.6
<i>edit-jawiki</i>	2.981M	42.83M	385K	3.5	7.0	3.0	2.4	1.3	2.9
<i>socfb-B-anon</i>	2.937M	41.91M	4356	2.5	5.0	2.3	1.8	1.3	2.7
<i>edit-plwiki</i>	2.646M	42.43M	850K	11.9	19.6	9.9	7.6	1.3	2.6
<i>edit-ukwiki</i>	2.132M	17.44M	306K	1.8	2.5	1.6	1.2	1.3	2.0
<i>soc-pokec</i>	1.632M	44.60M	14.8K	1.7	3.2	1.7	1.0	1.7	3.3

Table 2: Performance Comparison on Subgraph Matching (SM)

(a) Query Graphs:

(b) Degree Distributions:

(c) Time:

G	Query	G ² -AIMD	CPU-Only	GPU-Only	Hybrid	Gain ^{92G}	Gain ^{92C}	STMatch	EGSM	GAMMA
<i>soc-sinaweibo</i>	P1	35.5	OOT	58.4	59.2	1.0	—	Error	OOM	OOM
	P2	292.1	OOT	327.7	322.1	1.0	—	Error	OOM	OOM
	P3	470.7	OOT	431.6	436.7	1.0	—	Error	OOM	OOM
<i>edit-cebwiki</i> ③	P1	222.2	65.7	5.3*	5.3*	1.0	12.50	633.9	35.7	312.3
	P2	361.0	OOT	26.0*	26.0*	1.0	—	OOT	71.6	OOM
	P3	OOT	OOT	115.2	115.0	1.0	—	OOT	Error	OOM
<i>edit-shwiki</i> ④	P1	439.2	24.3	4.5*	4.5*	1.0	5.40	106.7	65.2	152.5
	P2	728.1	18.9	0.9*	0.9*	1.0	21.52	151.8	9.7	214.1
	P3	OOT	OOT	1165.2	1162.8	1.0	—	198.4	OOM	OOM
<i>edit-mgwiktionary</i>	P1	0.6	141.5	0.3	0.7	0.5	204.74	OOT	11.8	759.9
	P2	0.8	30.2	0.3	0.5	0.6	64.02	710.1	4.6	549.9
	P3	10.0	213.3	0.5	6.9	0.1	30.92	OOT	13.1	554.4
<i>soc-livejournal</i>	P1	2.2	8.3	2.0	2.3	0.9	3.59	26.9	OOM	16.5
	P2	61.3	476.8	51.7	49.9	1.0	9.56	OOT	OOM	OOM
	P3	102.3	767.5	94.6	94.5	1.0	8.13	OOT	OOM	OOM
<i>socfb-B-anon</i> ①	P1	1.7	5.7	1.5	1.7	0.9	3.28	7.4	22.6	14.5
	P2	14.1	34.2	10.2	10.2	1.0	3.35	32.9	OOM	OOM
	P3	31.5	157.2	22.2	22.6	1.0	6.96	31.7	OOT	OOM
<i>soc-pokec</i> ②	P1	2.0	2.8	1.8	1.0	1.7	2.68	13.1	12.1	7.8
	P2	6.5	9.5	6.0	4.0	1.5	2.38	9.1	196.7	24.2
	P3	13.8	53.2	7.7	7.9	1.0	6.70	11.2	OOM	106.6

throughput. Based on this observation, we recommend not to enable CPU processing in G-ThinkerCG for the SM application.

We also include the latest GPU solutions to SM for comparison, STMatch [36], EGSM [30] and GAMMA [22], the results of which are shown in the last three columns of Table 2(c). We can see that all three systems are much slower than GPU-Only and Hybrid, which shows the superiority of our BFS-DFS hybrid strategy for subgraph extension on a GPU.

System Parameter Study. Due to space constraints, we report a detailed system parameter study in Appendix E [1]. Overall, the default parameter settings provide robust performance in general.

8 CONCLUSIONS AND FUTURE WORK

We presented G-thinkerCG, a general-purpose programming framework for subgraph finding (SF) that enables unified CPU-GPU co-processing. G-thinkerCG combines a task-centric programming model with a BFS-DFS hybrid execution scheme on the GPU, allowing it to effectively handle highly irregular and straggler-prone SF workloads. Experimental results demonstrate that G-thinkerCG outperforms CPU-only and GPU-only approaches and achieves efficient CPU-GPU collaboration.

As future work, we plan to extend G-thinkerCG to support additional SF applications such as quasi-cliques and k -plexes, as well as to explore multi-GPU and distributed deployments.

REFERENCES

- [1] G-thinkerCG online appendix. <https://anonymous.4open.science/r/G-ThinkerCG-AFD6/appendix.pdf>
- [2] Network Datasets from KONECT. <http://konect.cc/networks/>.
- [3] Network repository. <https://networkrepository.com>.
- [4] A. Ahmad, D. Yan, X. Chen, L. Yuan, Q. Zhang, and S. Adhikari. Maximum k -plex finding: Choices of pruning techniques matter! *Proc. VLDB Endow.*, 18(9):2928–2940, 2025.
- [5] V. Batagelj and M. Zaversnik. An $o(m)$ algorithm for cores decomposition of networks. *CoRR*, cs.DS/0310049, 2003.
- [6] C. Bron and J. Kerbosch. Finding all cliques of an undirected graph (algorithm 457). *Commun. ACM*, 16(9):575–576, 1973.
- [7] L. Chang, M. Xu, and D. Strash. Efficient maximum k -plex computation over large sparse graphs. *Proc. VLDB Endow.*, 16(2):127–139, 2022.
- [8] H. Chen, M. Liu, Y. Zhao, X. Yan, D. Yan, and J. Cheng. G-miner: an efficient task-oriented graph mining system. In *EuroSys*, pages 32:1–32:12. ACM, 2018.
- [9] J. Chen, S. Cai, S. Pan, Y. Wang, Q. Lin, M. Zhao, and M. Yin. Nuqclq: An effective local search algorithm for maximum quasi-clique problem. In *AAAI*, pages 12258–12266. AAAI Press, 2021.
- [10] J. Chen and X. Qian. Khuzdul: Efficient and scalable distributed graph pattern mining engine. In T. M. Aamodt, N. D. E. Jerger, and M. M. Swift, editors, *ASPLOS*, pages 413–426. ACM, 2023.
- [11] X. Chen, R. Dathathri, G. Gill, and K. Pingali. Pangolin: An efficient and flexible graph mining system on CPU and GPU. *Proc. VLDB Endow.*, 13(8):1190–1205, 2020.
- [12] Q. Cheng, D. Yan, T. Wu, L. Yuan, J. Cheng, Z. Huang, and Y. Zhou. Efficient enumeration of large maximal k -plexes. In A. Simitis, B. Kemme, A. Queral, O. Romero, and P. Jovanovic, editors, *EDBT*, pages 53–65. OpenProceedings.org, 2025.
- [13] A. Ching, S. Edunov, M. Kabiljo, D. Logothetis, and S. Muthukrishnan. One trillion edges: Graph processing at facebook-scale. *Proc. VLDB Endow.*, 8(12):1804–1815, 2015.

- [14] P. Cui, H. Liu, B. Tang, and Y. Yuan. Cggraph: An ultra-fast graph processing system on modern commodity CPU-GPU co-processor. *Proc. VLDB Endow.*, 17(6):1405–1417, 2024.
- [15] Q. Dai, R. Li, H. Qin, M. Liao, and G. Wang. Scaling up maximal k -plex enumeration. In M. A. Hasan and L. Xiong, editors, *CIKM*, pages 345–354. ACM, 2022.
- [16] V. V. dos Santos Dias, C. H. C. Teixeira, D. O. Guedes, W. M. Jr., and S. Parthasarathy. Fractal: A general-purpose graph pattern mining system. In *SIGMOD*, pages 1357–1374. ACM, 2019.
- [17] S. Gao, K. Yu, S. Liu, and C. Long. Maximum k -plex search: An alternated reduction-and-bound method. *Proc. VLDB Endow.*, 18(2):363–376, 2024.
- [18] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin. Powergraph: Distributed graph-parallel computation on natural graphs. In C. Thekkath and A. Vahdat, editors, *OSDI*, pages 17–30. USENIX Association, 2012.
- [19] M. Greenbaum, K. Fan, W. Manzoor, and G. Guo. Gpu-accelerated maximal quasi-clique mining. In *IEEE BigData*, pages 152–161. IEEE, 2025.
- [20] G. Guo, D. Yan, M. T. Özsu, Z. Jiang, and J. Khalil. Scalable mining of maximal quasi-cliques: An algorithm-system codesign approach. *Proc. VLDB Endow.*, 14(4):573–585, 2020.
- [21] G. Guo, D. Yan, L. Yuan, J. Khalil, C. Long, Z. Jiang, and Y. Zhou. Maximal directed quasi-clique mining. In *ICDE*, pages 1900–1913. IEEE, 2022.
- [22] L. Hu, L. Zou, and M. T. Özsu. Gamma: A graph pattern mining framework for large graphs on gpu. In *ICDE*. IEEE, 2023.
- [23] K. Jamshidi, R. Mahadasa, and K. Vora. Peregrine: a pattern-aware graph mining system. In *EuroSys*, pages 13:1–13:16. ACM, 2020.
- [24] J. Khalil, D. Yan, G. Guo, and L. Yuan. Parallel mining of large maximal quasi-cliques. *VLDB J.*, 31(4):649–674, 2022.
- [25] G. Liu and L. Wong. Effective pruning techniques for mining quasi-cliques. In *PKDD*, volume 5212 of *Lecture Notes in Computer Science*, pages 33–49. Springer, 2008.
- [26] Y. Lu, J. Cheng, D. Yan, and H. Wu. Large-scale distributed graph computing systems: An experimental evaluation. *Proc. VLDB Endow.*, 8(3):281–292, 2014.
- [27] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski. Pregel: a system for large-scale graph processing. In *SIGMOD Conference*, pages 135–146. ACM, 2014.
- [28] A. Maratea, L. Marcellino, and V. Duraccio. A gpu-parallel algorithm for fast hybrid BFS-DFS graph traversal. In *SITIS*, pages 450–457. IEEE Computer Society, 2017.
- [29] S. Sun and Q. Luo. In-memory subgraph matching: An in-depth study. In *SIGMOD*, pages 1083–1098, 2020.
- [30] X. Sun and Q. Luo. Efficient gpu-accelerated subgraph matching. *Proc. ACM Manag. Data*, 1(2):181:1–181:26, 2023.
- [31] Y. Sun, J. Zhang, H. Cao, Y. Zhang, X. An, J. Huang, and X. Ye. Cggraph: Efficient CPU-GPU co-execution for concurrent dynamic graph processing. *ACM Trans. Archit. Code Optim.*, 22(3):91:1–91:26, 2025.
- [32] C. H. C. Teixeira, A. J. Fonseca, M. Serafini, G. Siganos, M. J. Zaki, and A. Aboul-naga. Arabesque: a system for distributed graph mining. In *SOSP*, pages 425–440. ACM, 2015.
- [33] J. R. Ullmann. An algorithm for subgraph isomorphism. *J. ACM*, 23(1):31–42, 1976.
- [34] K. Wang, Z. Zuo, J. Thorpe, T. Q. Nguyen, and G. H. Xu. Rstream: Marrying relational algebra with streaming for efficient graph mining on a single machine. In *OSDI*, pages 763–782. USENIX Association, 2018.
- [35] Z. Wang, Y. Zhou, M. Xiao, and B. Khousainov. Listing maximal k -plexes in large real-world graphs. In F. Laforest, R. Troncy, E. Simperl, D. Agarwal, A. Gionis, I. Herman, and L. Médini, editors, *WWW*, pages 1517–1527. ACM, 2022.
- [36] Y. Wei and P. Jiang. Stmatch: Accelerating graph pattern matching on GPU with stack-based loop optimizations. In F. Wolf, S. Shende, C. Culhane, S. R. Alam, and H. Jagode, editors, *SC*, pages 53:1–53:13. IEEE, 2022.
- [37] D. Yan, Y. Bu, Y. Tian, and A. Deshpande. Big graph analytics platforms. *Found. Trends Databases*, 7(1-2):1–195, 2017.
- [38] D. Yan, J. Cheng, K. Xing, Y. Lu, W. Ng, and Y. Bu. Pregel algorithms for graph connectivity problems with performance guarantees. *Proc. VLDB Endow.*, 7(14):1821–1832, 2014.
- [39] D. Yan, G. Guo, M. M. R. Chowdhury, M. T. Özsu, W. Ku, and J. C. S. Lui. G-thinker: A distributed framework for mining subgraphs in a big graph. In *ICDE*, pages 1369–1380. IEEE, 2020.
- [40] D. Yan, G. Guo, J. Khalil, M. T. Özsu, W. Ku, and J. C. S. Lui. G-thinker: a general distributed framework for finding qualified subgraphs in a big graph with load balancing. *VLDB J.*, 31(2):287–320, 2022.
- [41] K. Yao and L. Chang. Efficient size-bounded community search over large networks. *Proc. VLDB Endow.*, 14(8):1441–1453, 2021.
- [42] C. Ye, Y. Li, S. Sun, and W. Guo. gsword: Gpu-accelerated sampling for subgraph counting. *Proc. ACM Manag. Data*, 2(1):33:1–33:26, 2024.
- [43] L. Yuan, A. Ahmad, D. Yan, J. Han, S. Adhikari, X. Yu, and Y. Zhou. G²-aimd: A memory-efficient subgraph-centric framework for efficient subgraph finding on gpus. In *ICDE*, pages 3164–3177. IEEE, 2024.
- [44] L. Yuan, G. Guo, D. Yan, S. Adhikari, J. Khalil, C. Long, and L. Zou. G-thinkerq: A general subgraph querying system with a unified task-based programming model. *IEEE Trans. Knowl. Data Eng.*, 37(6):3429–3444, 2025.
- [45] L. Yuan, D. Yan, J. Han, A. Ahmad, Y. Zhou, and Z. Jiang. Faster depth-first subgraph matching on gpus. In *ICDE*, pages 3151–3163. IEEE, 2024.

Algorithm 3 Ullmann’s Subgraph Matching Algorithm

- 1: **Input:** query graph $G_q = (V_q, E_q)$, data graph $G = (V, E)$
- 2: generate matching order $\pi = [u_1, u_2, \dots, u_{|V_q|}] \quad (u_i \in V_q)$
- 3: **ENUMERATE**($\emptyset, 1, \pi$)
- 4: **procedure** **ENUMERATE**(S, i, π)
- 5: $C_S(u_i) \leftarrow$ viable vertex candidates in G to match u_i
- 6: **for all** $v \in C_S(u_i)$ **do**
- 7: append S with (u_i, v)
- 8: **if** $|S| = k$ **then output** S
- 9: **else** **ENUMERATE**($S, i + 1, \pi$)
- 10: **pop** (u_i, v) from S

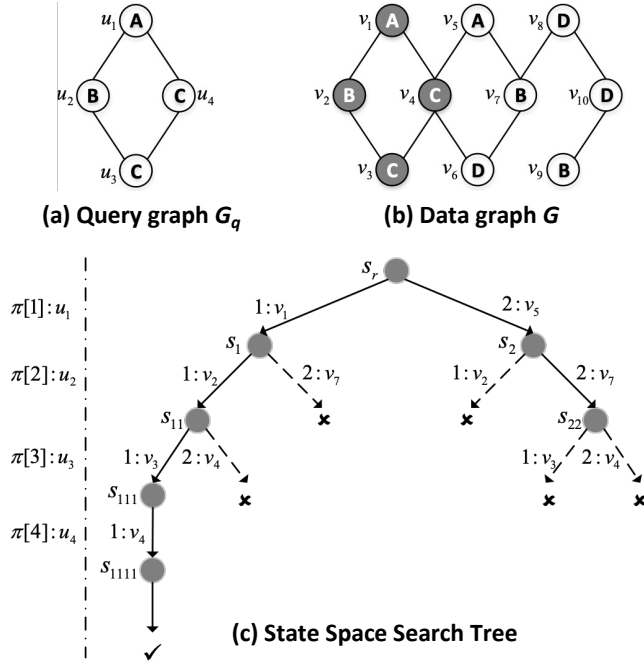


Figure 9: Illustration of Subgraph Matching

A SUBGRAPH MATCHING

Subgraph matching (SM) is a representative SF problem that enumerates all subgraphs of a data graph G that are isomorphic to a query graph G_q . Many SM algorithms [29, 44] are based on Ullmann’s backtracking framework [33], which orders query vertices and recursively extends a partial mapping while pruning infeasible candidates.

For completeness, we outline a basic Ullmann-style backtracking procedure in Algorithm 3. The algorithm incrementally matches data vertices to query vertices $u_1, \dots, u_{|V_q|}$ according to a predefined matching order, maintaining the current partial mapping in S . At each recursion level, viable candidate vertices in G are derived for the next query vertex, and infeasible extensions are pruned early based on adjacency constraints. The search proceeds recursively until either a complete match is found or no further extension is possible.

This process explores a state-space tree as illustrated in Figure 9. A node at level i corresponds to a partial mapping that matches the

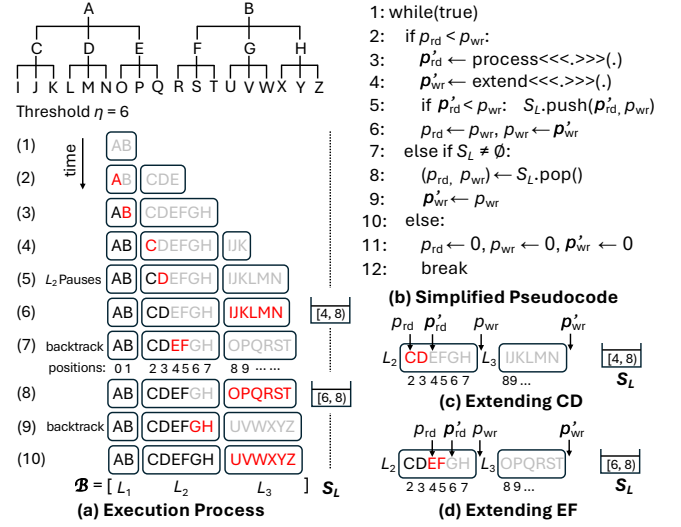


Figure 10: Example of Hybrid BFS-DFS Subgraph Extension

query vertex sequence prefix $[u_1, \dots, u_i]$. For example, the partial mapping $S = [(u_1, v_5), (u_2, v_7), (u_3, v_4)]$ is pruned due to the absence of an edge (v_7, v_4) corresponding to the query edge (u_2, u_3) . Various optimizations adopted by modern SM algorithms [29]—such as improving the matching order (Line 2) and tightening candidate sets (Line 5)—aim to reduce the search space but preserve this fundamental backtracking structure.

Importantly, the state-space tree explored by Ullmann-style algorithms is also a subgraph enumeration tree over G : each node represents a subgraph in G that matches a prefix of the query graph.

B GPU WORKER BACKTRACKING

Figure 10(b) presents the GPU worker’s backtracking control loop for implementing the hybrid BFS-DFS extension described in Section 3.1. Each iteration of the loop extends subgraphs from the current level L_i to the next level L_{i+1} .

For ease of presentation, let us continue to assume that each subgraph occupies only one logical position in the device buffer $\mathcal{B}$, and Appendix C will generalize it to the actual variable-length subgraph representation.

We represent L_i as a contiguous segment $\mathcal{B}[p_{rd}, p_{wr}]$ in $\mathcal{B}$. Newly generated subgraphs of L_{i+1} are appended starting at p_{wr} , with p'_{wr} tracking the current end of the output segment. Meanwhile, p'_{rd} tracks the next parent in L_i to be processed: $\mathcal{B}[p_{rd}, p'_{rd}]$ has been processed, while $\mathcal{B}[p'_{rd}, p_{wr}]$ remains unprocessed. The evolution of these pointers is illustrated in Figure 10(c)–(d).

Since multiple GPU warps concurrently consume parents from L_i and append children to L_{i+1} , p'_{rd} and p'_{wr} act as shared read and write pointers, respectively, and are advanced atomically as warps acquire parents and reserve output positions.

We now explain the control loop of Figure 10(b). At Line 2, the GPU worker first checks whether the current level $L_i = \mathcal{B}[p_{rd}, p_{wr}]$ is non-empty. If so, it invokes the application UDFs in Lines 3–4 to process and extend the current level.

The runtime supports two UDF execution modes. When the optional `process(.)` UDF is provided, GPU warps independently

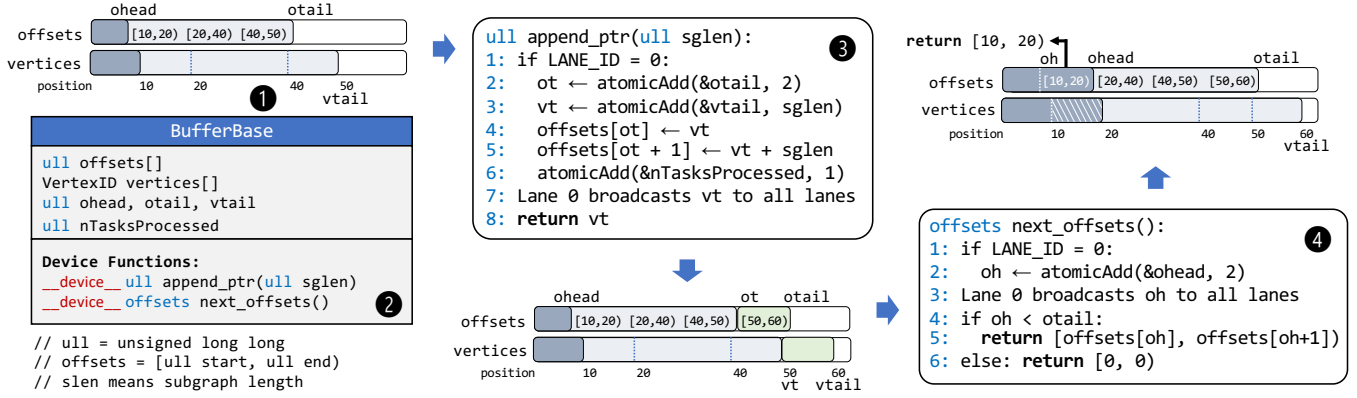


Figure 11: BufferBase and Its Operations

acquire parent subgraphs from L_i and generate their candidate elements into an auxiliary device buffer. The runtime stops assigning new parents once either all parents have been examined or the per-round expansion threshold η has been reached. Upon return, p'_{rd} indicates the boundary between processed and unprocessed parents. The subsequent `extend(.)` invocation materializes child subgraphs from the generated candidates and appends them to L_{i+1} . Making candidates explicit allows work to be distributed at candidate granularity, improving load balance when different parents have highly skewed candidate counts.

Alternatively, an application may implement `extend(.)` alone. In this mode, each warp acquires a parent, generates its candidates on the fly, and directly materializes the corresponding child subgraphs. This avoids the intermediate candidate buffer, although work distribution is then performed at parent granularity and may be less balanced under highly skewed candidate counts. In both modes, the runtime monitors the expansion threshold η and stops assigning new parents once the threshold is reached. Because multiple warps execute concurrently, warps already processing assigned parents may still append their remaining children, so the actual number of generated subgraphs may slightly exceed η .

After the UDF invocation completes, Line 5 records any unprocessed parents in L_i . Specifically, if $p'_{rd} < p_{wr}$, the GPU worker pushes $\langle p'_{rd}, p_{wr} \rangle$ onto the backtracking stack S_L . This pair compactly represents the deferred portion of the current level without copying its subgraphs. The worker then advances to L_{i+1} in Line 6 by setting $p_{rd} \leftarrow p_{wr}$ and $p_{wr} \leftarrow p'_{wr}$. The next iteration therefore continues with the newly generated frontier. For example, Figure 10(c) shows that after extending C and D , the remaining range $[4, 8)$ of L_2 is pushed onto S_L before execution proceeds to L_3 .

When the current level becomes empty, the worker checks S_L in Line 7. If deferred work exists, Line 8 pops the most recently suspended range and restores it as the current level. The output boundary is reset in Line 9 so that this range can generate a fresh next-level segment. Figure 10(d) illustrates this process: after E and F are extended, the remaining range $[6, 8)$ is deferred and later restored to resume extension of G and H .

Finally, if both the current level and S_L are empty, all subgraphs reachable from the current input chunk have been processed. The

worker therefore resets $\mathcal{B}$ (Line 11) and terminates the backtracking loop (Line 12).

C SUBGRAPH BUFFER IMPLEMENTATION

The device buffer $\mathcal{B}$ and host spill stack $\mathcal{H}$ store variable-length subgraphs, each represented by a sequence of vertex IDs. To support compact storage and efficient movement of subgraphs between device and host, G-ThinkerCG uses a common data structure, BufferBase, whose device-side layout and operations are illustrated in Figure 11. Using the same representation for $\mathcal{B}$ and $\mathcal{H}$ allows the GPU worker to transfer batches of subgraphs between them without application-specific serialization or conversion.

Buffer Representation. As illustrated by ① and ② in Figure 11, a BufferBase consists of two preallocated arrays: `vertices`, which stores the vertex IDs of subgraphs contiguously, and `offsets`, which stores a pair $[start, end)$ for each subgraph to identify its segment in `vertices`. This separation packs variable-length subgraphs compactly while keeping their descriptors fixed in size.

Three indices maintain the buffer state: `ahead` points to the next subgraph to be acquired for processing, `otail` marks the end of the subgraph descriptors in `offsets`, and `vtail` marks the end of the used region in `vertices`.

The buffer also maintains a counter `nTasksProcessed`, which tracks the number of subgraphs appended during the current expansion and is used to enforce the per-level threshold η .

Appending Subgraphs. Newly generated subgraphs are appended at warp granularity using `append_ptr(sglen)`, illustrated by ③ in Figure 11. The first lane of a warp atomically advances `otail` and `vtail` to reserve one descriptor and `sglen` consecutive entries in `vertices`, respectively (Lines 2–3), and records the resulting $[start, end)$ range in `offsets` (Lines 4–5). It also increments `nTasksProcessed` (Line 6), allowing the runtime and UDFs to detect when the expansion threshold η has been reached.

The reserved start position is then broadcast to all lanes (Line 7), after which the threads of the warp can cooperatively copy the subgraph’s vertex IDs into the reserved segment of `vertices` in parallel. Since space reservation is performed atomically, multiple warps can concurrently append variable-length subgraphs without overlapping their storage.

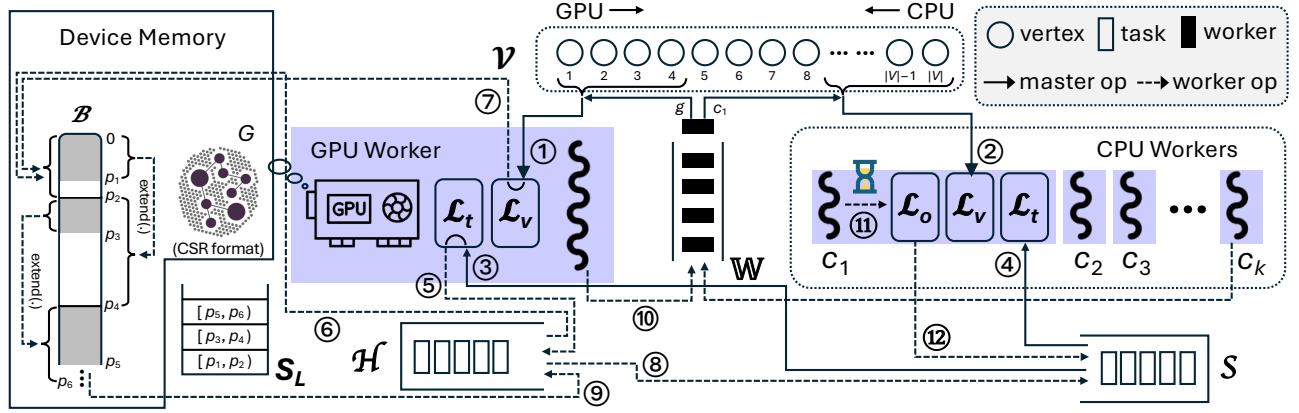


Figure 12: System Architecture and Task Flows in G-ThinkerCG

Fetching Subgraphs. Warps acquire subgraphs for extension using `next_offsets()`, illustrated by ④ in Figure 11. The first lane atomically advances ahead to claim the next descriptor (Line 2) and broadcasts the claimed position to all lanes (Line 3). If the position precedes `otail`, the primitive returns the corresponding `[start, end)` range (Lines 4–5), after which the warp cooperatively reads and processes the subgraph stored in vertices; otherwise, it returns an empty range (Line 6), indicating that no unclaimed subgraphs remain in the buffer. Since each descriptor position is claimed through an atomic operation, each subgraph is acquired by at most one warp even when many warps request work concurrently.

Together, `append_ptr()` and `next_offsets()` provide the device-side buffer interface exposed to application UDFs: user-written UDFs invoke these primitives to append newly generated subgraphs and fetch subgraphs for processing, while concurrent buffer book-keeping is handled transparently by the runtime.

Interaction with Host Buffering. The host spill stack $\mathcal{H}$ uses the same subgraph representation. As described in Section 3.2, when device space becomes insufficient, generated subgraphs are deferred to $\mathcal{H}$ for later processing. Once the current input chunk is exhausted, the GPU worker retrieves up to η most recently spilled subgraphs from the tail of $\mathcal{H}$ and transfers them into $\mathcal{B}$ as the next input batch. The common `offsets + vertices` representation enables such transfers without application-specific conversion, after which the restored subgraphs can be processed normally through `next_offsets()`.

D GPU WORKER DESIGN AND INTERFACE

This appendix details the implementation and control flow of the GPU worker summarized in Section 4.2. The GPU worker is a dedicated host-side CPU thread that launches GPU kernels and manages both the device execution buffer $\mathcal{B}$ and the host-resident task stack $\mathcal{H}$. It also serves as the boundary between the master-managed scheduling structures and GPU execution: tasks assigned by the master are delivered through the worker’s local buffers, while the GPU worker exclusively manages $\mathcal{B}$ and $\mathcal{H}$.

Algorithm 4 summarizes its control flow, and Figure 12 illustrates the corresponding data movement.

Algorithm 4 Pseudocode of the GPU Worker

```

1: while true do
2:   wait to be notified by master
3:   if global_end_flag = true then
4:     break
5:   if  $\mathcal{L}_t \neq \emptyset$  then
6:      $\mathcal{H} \leftarrow \text{moveTasksFromCPU}(\mathcal{L}_t)$  ▷ ⑤
7:      $v_{pos} \leftarrow 0$  ▷ next vertex in  $\mathcal{L}_v$ 
8:     while true do
9:       if  $\mathcal{H} \neq \emptyset$  then
10:        Pop up to  $\eta$  tasks from  $\mathcal{H}$  to  $\text{Bwr} (= \mathcal{L}_1)$  ▷ ⑥
11:        if  $|\mathbb{W}| > \tau_c$  then
12:           $\mathcal{S} \leftarrow \text{moveTasksToCPU}(\mathcal{H})$  ▷ ⑧
13:        else if  $v_{pos} < |\mathcal{L}_v|$  then
14:           $\text{Bwr}, v_{pos} \leftarrow \text{generateInitialTasks} \llcorner \llcorner \llcorner (\mathcal{L}_v, v_{pos})$  ▷ ⑦
15:        else
16:          break
17:         $\text{Brd} \leftarrow \text{Bwr}$  (i.e.,  $\mathcal{L}_1$ ); set  $\text{Bwr}$  to  $\mathcal{L}_2$ 
18:        run the hybrid BFS-DFS procedure in Figure 10(b) to extend
19:         $\text{Brd} = \mathcal{L}_1$  ▷ clear the iterator for correct invocation next time
20:         $\mathcal{L}_v \leftarrow \emptyset$ 
21:        append itself to  $\mathbb{W}$  and notify the master ▷ ⑩

```

Task Handoff and Buffer Ownership. When the master assigns subgraph-tasks from the global stack $\mathcal{S}$ to the GPU worker, it places them in the worker’s local task buffer $\mathcal{L}_t$ (Figure 12-③; Algorithm 2, Line 9). The GPU worker then converts these CPU-side tasks into the GPU-side subgraph representation through `moveTasksFromCPU(.)` and places them in $\mathcal{H}$ (Figure 12-⑤; Algorithm 4, Line 6). Tasks in $\mathcal{H}$ are subsequently transferred in batches into the device execution buffer $\mathcal{B}$ for GPU processing (Figure 12-⑥). Thus, the master never directly accesses either $\mathcal{H}$ or $\mathcal{B}$; it communicates with the GPU worker only through $\mathcal{L}_v$ and $\mathcal{L}_t$. This ownership boundary allows the GPU worker to manage its host- and device-side execution state without fine-grained synchronization with the master.

The stack $\mathcal{H}$ serves two related purposes. First, it receives tasks assigned to the GPU from $\mathcal{S}$ through $\mathcal{L}_t$. Second, as described in Section 3.2, it stores subgraphs spilled during GPU execution when

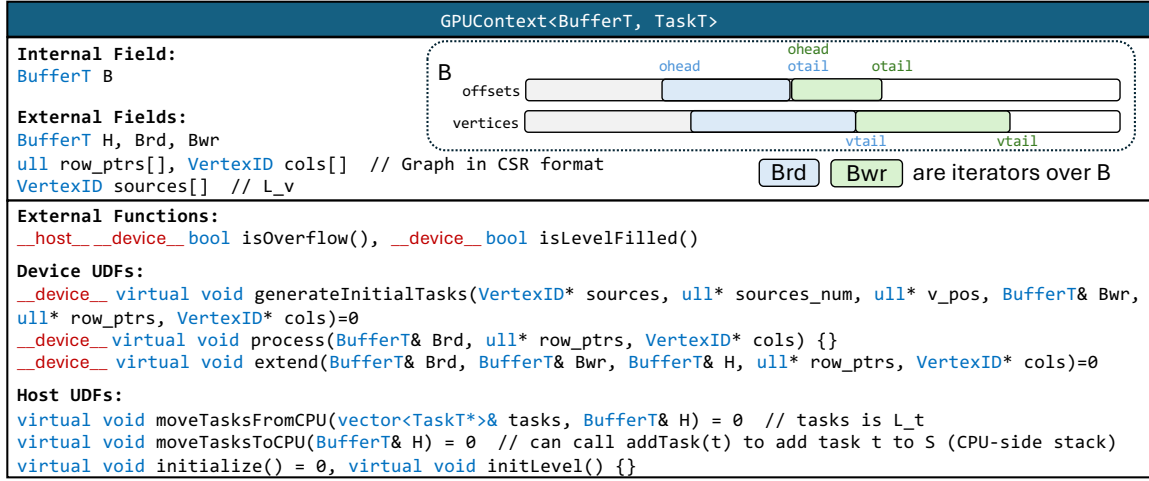


Figure 13: The GPUContext class with UDFs

$\mathcal{B}$ approaches capacity. The GPU worker processes tasks already in $\mathcal{H}$ before generating new tasks from $\mathcal{L}_v$, preserving the near-DFS scheduling order described in Section 3.2 and preventing pending work in $\mathcal{H}$ from accumulating.

GPUContext and Execution Buffers. Figure 13 shows the GPUContext maintained by the GPU worker. It encapsulates the GPU-side execution state, graph data, buffers, and application UDFs required for subgraph enumeration. Its central data structure is the device execution buffer $\mathcal{B}$ described in Section 3, which is logically organized into consecutive enumeration levels. During an extension from L_i to L_{i+1} , two BufferT views are maintained over $\mathcal{B}$: Brd identifies the current level L_i , from which UDF kernels acquire parent subgraphs using BufferT::next_offsets(), whereas Bwr identifies L_{i+1} , into which newly generated subgraphs are appended using BufferT::append_ptr(). These primitives are detailed in Appendix C.

The GPUContext also maintains $\mathcal{H}$ using the same BufferT representation. The storage of $\mathcal{H}$ is allocated in CUDA managed memory so that overflowing GPU kernels can directly append subgraphs to it, while the GPU worker can subsequently manipulate the same buffer from the host. The array sources[] provides the GPU-side view of the current vertex chunk in $\mathcal{L}_v$ and is consumed by generateInitialTasks(.) when creating the first enumeration level.

Initial-Task Generation. A GPU activation may start from either subgraph-tasks already present in $\mathcal{H}$ or initial vertices in $\mathcal{L}_v$. The former are given priority. If $\mathcal{H}$ is non-empty, the GPU worker transfers up to η tasks from its tail into Bwr, which becomes the first level L_1 of the next GPU execution batch (Algorithm 4, Lines 9–10). Otherwise, if unprocessed vertices remain in $\mathcal{L}_v$, the worker invokes generateInitialTasks(.) to construct L_1 directly from those vertices (Lines 13–14).

Since a GPU vertex chunk may contain substantially more than η vertices, generateInitialTasks(.) need not consume the entire $\mathcal{L}_v$ in one invocation. The variable v_{pos} records the next unprocessed vertex and is carried across successive invocations. Each

invocation generates a bounded batch of initial subgraphs and advances v_{pos} accordingly (Line 14). Once L_1 has been constructed, the worker sets Brd to this level, initializes Bwr for L_2 , and invokes the hybrid BFS-DFS extension procedure described in Section 3 and detailed in Appendix B (Lines 17–18). After that input batch is completely explored, the worker returns to $\mathcal{H}$ or $\mathcal{L}_v$ to construct the next batch.

Device UDFs. GPUContext exposes three application-defined device functions that specify GPU-side subgraph generation and extension:

- **generateInitialTasks(.)** creates initial subgraphs in Bwr ($= L_1$) from the vertices in $\mathcal{L}_v$ (Algorithm 4, Line 14; Figure 12-7). The parameter v_{pos} identifies the next unprocessed source vertex, allowing a large vertex chunk to be consumed over multiple bounded GPU batches.
- **process(.)** is optional and generates candidate elements for parent subgraphs acquired from Brd. Applications that use this two-stage execution mode store the candidates in an auxiliary device buffer $\mathcal{Q}$, which is allocated by the host UDF initialize() and reset by initLevel() before processing a new level. Separating candidate generation from subgraph materialization allows candidate-level work distribution when different parent subgraphs have highly skewed numbers of candidate elements for extension.
- **extend(.)** materializes child subgraphs and appends them to Bwr. When process(.) is provided, extend(.) consumes the candidates previously generated in $\mathcal{Q}$; otherwise, it generates candidates and materializes their extensions directly. If an output subgraph cannot be accommodated in the device buffer, it is redirected to $\mathcal{H}$ according to the host-buffering mechanism of Section 3.2.

These UDFs specify only application-dependent subgraph construction and pruning logic. Acquisition of parent subgraphs, reservation of output space, level transitions, backtracking, and overflow handling are provided by the runtime through the mechanisms described in Section 3 and Appendices B–C.

Expansion and Overflow Checks. Two runtime predicates coordinate bounded expansion with device-buffer capacity. Both predicates are exposed to application UDFs, allowing user-defined GPU code to query the runtime state during subgraph extension.

The device-side predicate `isLevelFilled(.)` indicates whether the current expansion should stop acquiring new parent subgraphs. It becomes true when either the per-round expansion threshold η has been reached, as tracked by `Bwr.nTasksProcessed`, or the device buffer is approaching capacity. As discussed in Section 3, η is a soft expansion bound because warps that have already acquired parents may finish generating their remaining children after the threshold is reached.

The predicate `isOverflow(.)` monitors the physical capacity of $\mathcal{B}$. Since `BufferT` separately stores subgraph descriptors in `offsets[]` and variable-length contents in `vertices[]`, exhaustion of either array constitutes an impending overflow. We conservatively regard the buffer as full when the utilization of either array exceeds 95%. This margin accommodates concurrent reservations by active warps while allowing most of the allocated device memory to remain usable.

During `extend(.)`, a child that cannot be safely appended to $\mathcal{B}$ is redirected to $\mathcal{H}$ rather than discarded. After the kernel returns, the GPU worker evaluates `isOverflow(.)` on the host. If overflow has been triggered, the portion of the output level already materialized in `Bwr` is also deferred to $\mathcal{H}$, freeing the device buffer so that the worker can backtrack and continue processing other resident work. This is the implementation-level realization of the host-buffering procedure illustrated in Figure 3.

CPU-GPU Task Conversion. The GPU and CPU implementations of an application may use different physical representations for the same logical subgraph-task. `GPUContext` therefore provides two host UDFs for crossing this representation boundary. The GPU worker invokes `moveTasksFromCPU(.)` when tasks assigned through $\mathcal{L}_t$ must be converted from `TaskT` objects into the `BufferT` representation used by $\mathcal{H}$. Conversely, `moveTasksToCPU(.)` converts tasks selected from $\mathcal{H}$ into CPU-side `TaskT` objects and inserts them into the global stack $\mathcal{S}$. These conversions are performed by the GPU worker rather than the master, preserving the ownership boundary around $\mathcal{H}$ and $\mathcal{B}$.

The second direction is activated only when CPU capacity is available. Specifically, when the number of idle CPU workers exceeds τ_c , the GPU worker may convert a batch of tasks from $\mathcal{H}$ and return them to $\mathcal{S}$ (Algorithm 4, Line 12; Figure 12-8). The master can subsequently assign these tasks to idle CPU workers. Together, $\mathcal{L}_t \rightarrow \mathcal{H}$ and $\mathcal{H} \rightarrow \mathcal{S}$ implement the two representation-conversion paths required by the bidirectional task redistribution described in Section 4.1.

GPU Worker Execution Flow. We finally summarize Algorithm 4. After being awakened by the master (Line 2), the GPU worker first checks the global termination flag (Lines 3–4) and imports any tasks assigned through $\mathcal{L}_t$ into $\mathcal{H}$ (Lines 5–6). It then repeatedly constructs an input batch, giving priority to pending tasks in $\mathcal{H}$ over new vertices in $\mathcal{L}_v$ (Lines 9–14). Each input batch is fully explored using the hybrid BFS-DFS procedure before the next batch is selected (Lines 17–18). During this process, newly generated

Table 3: Effect of System Parameters

Parameter	Value	soc-sinaweibo	wikipedia link sr
$\eta / \#\{\text{warps}\}$	100	203.8	OOT
	500	113.9	542.3
	1000	109.6	465.2
	2000	231.0	948.0
CPU Chunk	1	117.1	357.6
	10	108.9	316.1
	50	91.6	465.2
GPU Chunk	100	110.8	564.9
	100K	143.3	511.9
	500K	112.1	315.9
τ_c	1M	159.4	1085.5
	1	220.3	567.3
	$n_c / 2$	111.4	471.9
	$n_c / 4$	112.5	476.7

work may be spilled back to $\mathcal{H}$, and surplus tasks in $\mathcal{H}$ may be returned to $\mathcal{S}$ when CPU workers are underutilized (Lines 11–12).

The activation ends only when both $\mathcal{H}$ and the assigned vertex chunk $\mathcal{L}_v$ have been exhausted (Lines 15–16). The worker then clears $\mathcal{L}_v$, inserts itself into the idle-worker queue $\mathbb{W}$, and notifies the master (Lines 15–16). It remains idle until the master assigns another batch or sets the global termination flag (Lines 2–4).

E SYSTEM PARAMETER STUDY

We next present the effect of various system parameters on the running time of Hybrid using two representative datasets *soc-sinaweibo* and *wikipedia_link_sr*. As shown in Table 3 (where OOT means running out of the maximum allowed time threshold of 1800 seconds), our default parameters generally achieve the best performance. Note that a tradeoff exists for each parameter: when η and vertex chunk sizes are too large, the task granularity is too coarse which hampers load balancing; while when η and GPU chunk size are too small, the computing power of GPU threads cannot be saturated, and when CPU chunk size is too small, the cost for CPU workers to wait in $\mathbb{W}$ increases. As for τ_c , if it is too small, GPU-to-CPU task stealing would be too aggressive and could lead to task thrashing if some CPU worker encounters a task timeout and adds many new tasks back to $\mathcal{S}$; while if it is too large, surplus GPU tasks cannot be timely rescheduled to CPU workers for processing. Our default parameters achieve a good tradeoff.